КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА

Барабанов О.В.

Мова опису апаратних засобів VHDL

Навчальний посібник

Київ
Видавничо-поліграфічний центр
“Київський університет”
2002

О.В.Барабанов

Мова опису апаратних засобів VHDL: Навчальний посібник. – К.: ВПЦ “Київський університет”, 2002.- 82с.

В посібнику розглянуті основи мови VHDL, яка використовується при проектуванні цифрових електронних систем. Описані основні оператори мови, розглянуто їх синтаксис. Наведені приклади використання VHDL.

Видання розраховано на студентів старших курсів вищих учбових закладів; може бути цікавим фахівцям, які займаються проектуванням електронної апаратури.

© Барабанов О.В
© ВПЦ “Київський університет”, 2002

Передмова

Мова VHDL виникла в рамках виконання програми уряду США з розробки надшвидкодіючих інтегральних мікросхем (VHSIC – Very High Speed Integrated Circuits), розпочатої в 1980 році. У ході виконання цієї програми стало зрозумілим, що існує потреба в стандартній мові для опису структури та функції цифрових інтегральних мікросхем. Для цього була розроблена мова VHSIC Hardware Description Language (VHDL), яка у 1987 році була прийнята як стандарт Інститутом інженерів по електротехніці та електроніці (IEEE). Новий розширений стандарт цієї мови був прийнятий в 1993 році. Посібник спирається саме на цей стандарт, хоч більшість його положень стосуються і попереднього стандарту.

В передмові до стандартів IEEE вказано, що мова VHDL є "…формальним записом, який призначений для використання на всіх стадіях створення електронних систем… Вона підтримує розробку, верифікацію, синтез та тестування апаратного забезпечення." Мова VHDL дозволяє виконати декілька завдань, які виникають при проектуванні цифрових електронних систем. Вона дозволяє створити *структурний опис* системи, тобто описати, з яких модулів складається система та як ці модулі з'єднані між собою. Мова VHDL дозволяє створити *поведінковий опис* системи, в якому використовуються розповсюджені засоби алгоритмічних мов програмування – змінні, цикли, умовні оператори і т.і. Використання мови VHDL з відповідними програмними засобами дозволяє *промоделювати* систему без її виготовлення. Проектувальник може швидко порівнювати варіанти і перевіряти їх без створення апаратного макета. Після моделювання опис цифрової системи при наявності відповідного програмного забезпечення можна використати для *синтезу* реальної електронної схеми чи топології мікросхеми. Слід відзначити, що не всякий опис можна синтезувати. Детальний розгляд цього питання виходить за рамки даного посібника.

Цей посібник дозволяє познайомитися з мовою VHDL в об'ємі, достатньому для її практичного використання. Автор припускає, що читач знайомий з алгоритмічними мовами програмування, такими, як Паскаль, С чи Ада, та з основами

цифрової електроніки. Слід відзначити, що цей посібник не охоплює всіх положень мови VHDL. Опис синтаксису деяких операторів скорочено, а деякі зарезервовані слова не розглядаються зовсім. Повний опис мови наведений у відповідних стандартах.

У главі 1 розглянуті основні концепції мови VHDL. Ці концепції ілюструються простим прикладом. В цій главі описано використання нормальних форм Бекуса – Наура для запису синтаксичних правил, ці форми будуть застосовуватися в усіх главах посібника. Також в цій главі описані типи даних, змінні, константи та вирази мови VHDL. У главі 2 описані засоби, які використовуються для опису структури системи та її декомпозиції на підсистеми. В ній розглянуті об'єкти, архітектурні тіла, сигнали, компоненти. Глава 3 присвячена створенню поведінкового опису системи на мові VHDL. В ній описані алгоритмічні засоби мови VHDL, підпрограми, пакети та процеси. У главі 4 розглянуто, як всі описані раніше засоби використовуються для створення реальної моделі системи. Деякі додаткові питання розглянуті в главі 5.

1. Основи мови VHDL

1.1. Структурний опис цифрової системи

Однією з цілей розробки мови VHDL було створення засобу для моделювання роботи цифрової системи, тобто для розрахунку логічних станів сигналів в різні моменти часу. Перед моделюванням систему потрібно описати. Існує два підходи до опису цифрової системи – структурний та поведінковий. В цьому розділі ми розглянемо перший підхід, а в наступному – другий.

Цифрову електронну систему можна уявити собі як модуль з декількома входами та виходами. Сигнали на виходах являють собою деякі функції вхідних сигналів. На рис.1(а) наведено приклад цього представлення цифрової системи. Модуль M2 рахує суму двох двійникових розрядів. Він має два вході a та b і один вихід Y. У термінології VHDL модуль M2 є *об'єктом* (*entity*) проекту. Об'єкт проекту має вказані в його оголошенні входи та виходи, які називаються *портами* (*port*) і виконує описані в його *архітектурному тілі* (*architecture)* функції. Об'єкти проекту часто відповідають окремим функціонально-закінченим модулям, таким, як логічні елементи, мікросхеми, плати і т.д.

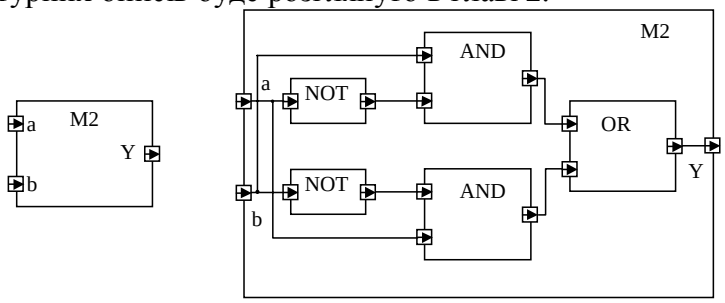
Один з можливих шляхів опису модуля полягає в тому, щоб описати, як цей об'єкт складається з екземплярів *компонентів* (*component*). Кожний з екземплярів компонентів – це деякий інший об'єкт, який описує модуль, що знаходиться на нижчому рівні ієрархії проекту. Порти екземплярів компонентів з'єднані між собою та з портами об'єкта верхнього рівня з використанням *сигналів* (*signal*). Перед створенням екземплярів компонент повинен бути оголошений в розділі оголошень архітектурного тіла. Сигнали, як і змінні, характеризуються типом, але, на відміну від змінних, мають інформацію про свій попередній стан та заплановані на майбутнє зміни своїх значень. Сигнали в VHDL відповідають електричним з'єднанням вузлів цифрової системи, яка розробляється.

Кожний об'єкт проекту може мати декілька архітектурних тіл. Яке з них потрібно використовувати при моделюванні та синтезі, які об'єкти використовувати при створенні екземплярів

компонентів та іншу додаткову інформацію вказують в оголошенні *конфігурації (configuration)*(див. розділ 4.2).

Оголошення об'єктів та конфігурації, архітектурні тіла, пакети та їх тіла (див. розділ 3.2.3) можуть розроблятися і аналізуватися незалежно, їх називають модулями проекту (див. розділ 4.1).

На рис.1 (б) показано, що об'єкт M2 може складатися з двох екземплярів компонента NOT, двох AND та одного OR. Такий опис називається *структурним (structural)*. Слід зауважити, що кожний з об'єктів NOT, AND, OR також може мати структурний опис, тобто складатися з інших компонентів і т.д. Створення структурних описів буде розглянуто в главі 2.



а б

Рис.1. Структурне представлення логічного елемента M2.

1.2. Поведінковий опис цифрової системи

Існує багато випадків, коли описувати об'єкт проекту структурно неможливо або недоцільно. Один такий випадок - об'єкт, який знаходиться у самому низу ієрархії деяких інших структурних описів. Інший випадок – розробка приладу з використанням готових мікросхем. Структурні описи мікросхем часто бувають дуже складні і моделювання приладів з таких елементів буде повільним. Внутрішня структура мікросхем, яка складає зміст структурного опису, може бути комерційною таємницею виробника. Також, часто при розробці системи буває доцільно спочатку описати функції, які виконують її модулі, промоделювати роботу системи, і тільки потім розробляти внутрішню структуру модулів. У таких випадках потрібен опис роботи об'єкта проекту, який не буде зв'язаним з його

внутрішньою структурою. Такий опис називають функціональним або *поведінковим* (*behavioral*).

Поведінковим описом об'єкта M2, який наведено на рис.1, може бути булева функція $Y = \bar{a}b \vee a\bar{b}$. У складніших випадках виходи об'єкта не можуть бути цілком описані як функція входів. У системах зі зворотним зв'язком виходи залежать також від попередніх станів системи. Алгоритм роботи таких систем в VHDL можна описати з використанням засобів, які притаманні алгоритмічним мовам програмування: змінних, циклів, умовних операторів і т.д. Ці оператори виконуються *послідовно*, як і в алгоритмічних мовах програмування. В главі 3 описані такі засоби мови VHDL.

Послідовні оператори виконуються в рамках одного чи декількох *процесів (process)* (див. розділ 3.3), які розташовані в архітектурних тілах. Самі процеси при моделюванні виконуються *паралельно*. Процеси можуть взаємодіяти між собою тільки за допомогою сигналів. В процесі можна присвоїти значення сигналу, який можна прочитати в іншому процесі. Процес має механізм призупинення його виконання та відновлення (див. розділ 3.3). Процес може призупинитися після виконання останнього оператора його тіла або спеціального оператора призупинення процесу. Робота процесу відновлюється або через вказаний проміжок часу, або коли змінюються вказані сигнали, або якщо їх значення задовольняють вказаній умові.

1.3. Модель дискретного часу

Якщо структура або поведінка об'єкта проекту визначена, то можна промоделювати його роботу. При моделюванні виконується розрахунок сигналів в дискретні моменти часу. Якщо в деякий момент часу один чи декілька вхідних сигналів об'єкта змінилися, то об'єкт реагує на ці зміни шляхом планування нових значень, що будуть присвоєні сигналам, які з'єднані з його виходами в трохи пізніший момент часу. Це називається плануванням *трансакції (transaction)* сигналу. Нові значення вихідних сигналів розраховуються шляхом виконання коду, який описує цей об'єкт. Кожна трансакція характеризується часом, на який вона запланована, та значенням, яке повинен прийняти сигнал в цей час. Множина запланованих трансакцій складає *драйвер (driver)* сигналу. Інформація про нову

трансакцію додається до драйвера сигналу оператором присвоювання значення сигналу, в якому вказується значення сигналу, яке планується, та затримка між часом, коли виконується цей оператор, та часом, на який буде запланована ця трансакція. Якщо затримка явно не вказана у операторі присвоювання значення сигналу, трансакція буде запланована на наступний крок моделювання. Така затримка називається дельта-затримкою. В будь-якому випадку трансакція виконується пізніше ніж оператор, який її планує.

На початку моделювання виконується ініціалізація, а потім повторюється цикл моделювання, який складається з двох стадій. При ініціалізації всім сигналам присвоюються початкові значення, час моделювання встановлюється на нуль, і один раз виконуються описи всіх об'єктів. Це звичайно приводить до планування трансакції хоч одного сигналу на пізніший момент часу.

На першій стадії циклу моделювання імітаційний час просувається до самого раннього часу, на який запланована будь-яка трансакція. Всі трансакції, які заплановані на цей час, виконуються, тобто сигналам присвоюються заплановані значення. Сигнал, для якого виконується трансакція, називають *активним (active)*. Якщо нове значення відрізняється від попереднього, сигнал змінюється, відбувається *подія (event)*, і інші об'єкти, для яких змінений сигнал є вхідним, активуються.

На другій стадії виконуються описи всіх об'єктів, які реагують на події, що відбулися на першій стадії. У більшості випадків це приведе до того, що на наступні моменти часу будуть заплановані трансакції деяких сигналів.

Коли всі описи виконані, цикл моделювання повторюється. Якщо більше немає запланованих трансакцій, моделювання закінчується.

Метою моделювання є отримання та збереження інформації відносно стану системи у будь-який момент часу. Також часто програма моделювання дозволяє виконувати покрокове моделювання, як це робиться відладчиком при програмуванні на алгоритмічних мовах програмування.

1.4. Приклад

У цьому розділі можливості мови VHDL будуть проілюстровані на прикладі створення опису RS-тригера (рис.2).

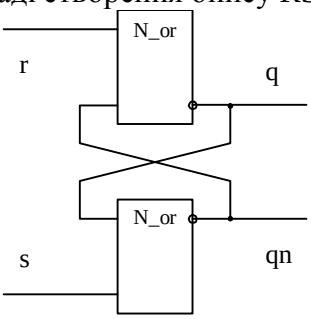


Рис.2. RS-тригер

Спочатку розглянемо об’єкт nor, який є логічним елементом АБО-НІ. Цей об’єкт буде використовуватися для створення структурного опису RS-тригера. Оголошення об’єкта є описом його зовнішнього інтерфейсу, тобто його портів та параметрів настроювання. Об’єкт nand може бути оголошений таким чином:

```
entity nor is
-- delay – параметр настроювання, який визначає швидкодію елемента
        generic(delay : time := 10 ns);
        port(   a : in bit; -- Входи
                b : in bit;
                c : out bit); -- Вихід
end nand;
```

В оголошенні об’єкта nor вказано, що він має два входи і один вихід, всі вони мають тип bit, тобто можуть приймати значення '0' або '1'. Також оголошено параметр настроювання delay, який використовується для керування роботою об’єкта (у цьому прикладі – це затримка сигналу). Якщо при створенні екземпляра цього об’єкта явно не вказано значення цього параметра, буде використовуватися значення по замовчуванню, в цьому прикладі 10 ns.

Реалізація об’єкта описана в його архітектурному тілі. Для одного об’єкта може існувати декілька архітектурних тіл, кожне з яких по різному описує структуру та поведінку об’єкта. Для об’єкта nand ми розглянемо одне архітектурне тіло, яке описує його поведінку:

```
architecture behaviour of nor is
begin
```

```
        c <= not(a or b) after delay;
end behaviour;
```

RS-тригер має два входи та два виходи. Відповідний об'єкт rstrg може бути оголошений таким чином:

```
entity rstrg is
        port(   r : in bit;
                s : in bit;
                q : out bit;
                qb : out bit);
end rstrg;
```

Ми розглянемо два архітектурних тіла цього об'єкта – поведінкове (behaviour) та структурне (structure). В архітектурному тілі можуть бути створені екземпляри компонентів та описані процеси.

```
architecture behaviour of rstrg is
        signal state : bit := '0';
begin
        rs_ff: process (r,s)
        begin
                if s='1' then
                        state <='1' ;
                end if;
                if r='1' then
                        state <= '0' ;
                end if;
        end process;
        q<= state after 5 ns;
        qb<=not state after 5 ns;
end behaviour;
```

У цьому архітектурному тілі оголошено внутрішній сигнал state, який зберігає стан тригера. Робота тригера описана в процесі, який називається rs_ff, в список чутливості якого входять два вхідних сигнали тригера – r та s. Процес виконується кожний раз, коли змінюється хоч один з сигналів, які входять до списку чутливості процесу. В самому процесі стоять умовні оператори, які перевіряють значення вхідних сигналів, та оператори присвоювання значення сигналу state. Після виконання останнього оператора процес призупиняється до моменту наступної зміни сигналу зі списку чутливості процесу. Останні два оператори змінюють вихідні порти об'єкта rstrg завжди, коли змінюється внутрішній сигнал state. В цих операторах вказана затримка 5 ns, це затримка між часом виконання оператора та часом, на який буде запланована трансакція вихідного сигналу.

Всі оператори (процес та два присвоювання) в архітектурному тілі паралельні, при моделюванні вони виконуються одночасно. Це поведінковий опис тригера, його робота описана за допомогою алгоритмічних засобів мови.

RS-тригер може бути реалізований у вигляді добре відомої схеми (див. рис.2), яка складається зі з'єднаних між собою двох елементів ТА-НІ. Таку схему також можна описати на мові VHDL:

```
architecture structure of rstrg is
        component nor           -- оголошення компонента nand
                generic(delay : time);
                port(   a : in bit;
                        b : in bit;
                        c : out bit);
        end component;
        -- оголошення внутрішніх сигналів
        signal st1 : bit := '0';
        signal st2 : bit := '1';
begin
        u1: nor -- створення екземпляра u1 компонента nor
                generic map(5 ns)
                -- специфікація параметра настроювання
                port map(a=>s, b=>st2, c=>st1);
                -- під'єднання сигналів до портів компонента
        u2: nor – створення екземпляра u2 компонента nor
                generic map(5 ns)
                port map(st1, r, st2);
        q<= st1;
        qb<= st2;
end structure;
```

В цьому архітектурному тілі оголошено компонент nor, а потім створено два екземпляри цього компонента, порти яких з'єднані з внутрішніми сигналами st1 та st2 та портами об'єкта rstrg. Наприклад, u1 – це екземпляр об'єкта nor, його порт a з'єднується з портом s об'єкта rstrg, а його порти b та c з'єднуються з внутрішніми сигналами st2 та st1. Відповідність імен портів та сигналів може бути іменованою, як для екземпляра u1, та позиційною, як для екземпляра u2. Такий опис називають структурним, оскільки він вказує, з яких компонентів складається об'єкт і як вони з'єднані між собою.

1.5. Нормальні форми Бекуса – Наура

Нормальні форми Бекуса – Наура використовуються для запису синтаксису мов програмування. Їх зміст описується наступними правилами:

а) Слова у нижньому регістрі, які інколи з'єднані символом підкреслювання, позначають синтаксичні категорії. Наприклад:

літерал_рядок_бітів

б) Жирний шрифт використовується для зарезервованих слів, наприклад:

type

в) Синтаксичне правило складається з синтаксичної категорії, символу "::=" та визначення цієї синтаксичної категорії. Наприклад:

повне_оголошення_типу ::= **type** ідентифікатор **is** визначення_типу;

Зміст правила полягає в можливості заміни синтаксичної категорії на її визначення.

г) Вертикальна лінія розділяє альтернативи, за винятком випадку, коли вона стоїть після відкриваючої фігурної дужки. Тоді вона спеціального значення не має. Наприклад:

розширена_цифра ::= цифра | буква
список_виборів ::= вибір { | вибір }

В першому прикладі синтаксична категорія "розширена_цифра" може бути замінена на синтаксичну категорію "цифра" чи "буква". В другому прикладі синтаксична категорія "список_виборів" може бути замінена на список категорій "вибір", елементи якого розділені вертикальними лініями.

е) Квадратні дужки позначають необов'язкові елементи. Наприклад, наступні два правила еквівалентні:

оператор_повернення ::= **return** [вираз];
оператор_повернення ::= **return**; | **return** вираз;

ж) Фігурні дужки позначають елементи, які можуть повторюватися 0, 1 або більше разів. Наприклад:

операторна_частина_архітектурного_тіла ::= { паралельний_оператор }

з) Якщо в назві синтаксичної категорії є частина, яка надрукована *курсивом*, то така категорія синтаксично еквівалентна категорії без цієї частини. Ця додаткова частина несе семантичну інформацію. Наприклад, синтаксичні категорії

інтерфейсний_список_*параметрів_настроювання*
інтерфейсний_список_*портів*

синтаксично еквівалентні категорії інтерфейсний_список.

і) Термін просте_ім'я використовується у випадках повторного використання вже використаного раніше ідентифікатора. Наприклад:

architecture ідентифікатор **of** назва_*об'єкта* **is**
    розділ_оголошень_архітектурного_тіла
    begin
        операторна_частина_архітектурного_тіла
    end [просте_ім'я_*архітектурного_тіла*] ;

В цьому прикладі ідентифікатор на початку опису і просте_ім'я_*архітектурного_тіла* в кінці співпадають.

1.6. Лексичні Елементи

1.6.1. Коментарі

Коментарі в VHDL починаються з двох суміжних дефісів (--) і продовжуються до кінця рядка. Вони не обробляються при моделюванні.

1.6.2. Ідентифікатори

Ідентифікатори в VHDL – це зарезервовані слова і назви, які визначив програміст. Вони повинні задовольняти правилу:

ідентифікатор ::= буква { [символ_підкреслення] буква_чи_цифра }

Зауважте, що регістр букв ігнорується, так що ідентифікатори triger та Triger не розрізняються. Символ підкреслення в ідентифікаторах має значення, так що This_Name та ThisName – різні ідентифікатори.

1.6.3. Числа

Числові літерали можуть бути записані у десятковому, двійниковому, восьмирічному чи шістнадцятирічному вигляді. Якщо літерал включає крапку, він представляє дійсне число, в протилежному випадку він представляє ціле число.

десятковий_літерал ::= ціле [. ціле] [порядок]
ціле ::= цифра { [символ_підкреслення] цифра}
порядок ::= E [+] ціле | E - ціле

Деякі приклади:

0	1	123_456_789	987E6	цілі числові літерали
0.0	0.5	2.718_28	12.4E-9	дійсні числові літерали

Для запису літералів можуть бути використані і інші основи, крім 10:

літерал_з_основою ::= основа # ціле_з_основою [. ціле_з_основою] #
[порядок]
основа ::= ціле
ціле_з_основою ::= розширена_цифра
{ [символ_підкреслення] розширена_цифра }
розширена_цифра ::= цифра | буква

Основа та порядок являють собою десяткові цілі числа. Порядок вказує ступінь основи, на яку множиться літерал. Символи від A до F (верхній чи нижній регістр) використовуються, щоб представити числа від 10 до 15. Наприклад:

2#1100_0100# 16#C4#	4#301#E1	ціле число 196
2#1.1111_1111_111#E+11	16#F.FF#E2	дійсне число 4095.0

1.6.4. Символи

Символьні літерали складаються з одного символу ASCII, взятого в одиночні лапки. Наприклад:

'A' '*' '"' ' '

1.6.5. Рядки

Рядкові літерали складаються з послідовності символів ASCII, взятої в подвійні лапки. Щоб включати подвійні лапки в рядок, потрібно поставити поряд два символи подвійних лапок. Рядок може бути присвоєний об'єкту, який є масивом символів. Приклади рядків:

"A string"	
""	пустий рядок
"A string in a string: ""A string"". "	рядок, який містить в собі подвійні лапки

1.6.6. Рядки бітів

В мові VHDL існує спеціальний засіб для створення літеральних масивів типу bit ('0' та '1', див. розділ 1.7.4). Синтаксис масивів бітів такий:

літерал_рядок_бітів ::= специфікатор_основи " бітова_величина "
специфікатор_основи ::= B | O | X
бітова_величина ::= розширена_цифра
{ [символ_підкреслення] розширена_цифра }

Специфікатор основи B вказує, що за ним записане двійникове число, O – восьмирічне, а X – шістнадцятирічне. Деякі приклади:

B"1010110"	довжина 7 біт
O"126"	довжина 9 біт, двійниковий еквівалент

	"001_010_110"
X"56"	довжина – 8 біт, двійниковий еквівалент "0101_0110"

1.7. Типи та об'єкти даних

В мові VHDL існує декілька основних, чи скалярних, типів. З елементів цих типів можуть бути сформовані складені типи. Скалярні типи включають в себе числові типи (цілі та з плаваючою точкою), фізичні типи та перерахування (включаючи перерахування символів). Складених типів в VHDL два – матриці і записи. У VHDL також існують вказівний тип access та файловий тип file, які не розглядаються в цьому посібнику. Тип даних визначається його оголошенням:

```
повне_оголошення_типу ::= type ідентифікатор is визначення_типу ;
визначення_типу ::=
        визначення_скалярного_типу
        | визначення_складеного_типу
        | визначення_вказівного_типу
        | визначення_файлового_типу
визначення_скалярного_типу ::=
        визначення_типу_перерахування | визначення_цілого_типу
        | визначення_типу_з_плаваючою_точкою
        | визначення_фізичного_типу
визначення_складеного_типу ::=
        визначення_індексного_типу
        | визначення_іменованого_типу
```

Приклади оголошень різних типів подаються в наступних розділах.

Мова VHDL строго типізована. Це означає, що компілятор не виконує ніяких неявних перетворень типів. Всі необхідні перетворення повинні бути виконані за допомогою явно викликаних підпрограм чи специфікаторів перетворення типу. При використанні специфікатора перетворення вираз, тип якого потрібно перетворити, беруть у дужки, а перед ним ставлять назву цільового типу. Наприклад, якщо змінні N_int та N_real мають відповідно цілий (див. розділ 1.7.1) та дійсний (розділ 1.7.3) типи, то оператор

```
N_int := integer(N_real);
```

дозволяє виконати перетворення між цими типами (звичайно, з втратою точності). Такі перетворення можна виконувати лише між скалярними арифметичними типами та їх масивами однакової розмірності.

1.7.1. Цілий тип

Множина значень, яка визначена *цілим типом*, являє собою множину цілих чисел у межах встановленого діапазону.

визначення_цілого_типу ::= обмеження_діапазону
обмеження_діапазону ::= **range** діапазон
діапазон ::= простий_вираз напрямок простий_вираз
напрямок ::= **to** | **downto**

Вирази, які визначають діапазон, повинні приймати цілі значення. Діапазони, які оголошені з напрямком **to**, називають *зростаючими діапазонами*, а діапазони, які оголошені з ключовим напрямком **downto**, називають *спадаючими діапазонами*. Стандарт VHDL обмежує лише мінімальний діапазон, який завжди можна реалізувати – від $-(2^{31}-1)$ до $2^{31}-1$.

Деякі приклади оголошень цілих типів:

```
type byte_int is range 0 to 255;
type signed_word_int is range -32768 to 32767;
type bit_index is range 31 downto 0;
```

У VHDL є напередвизначений цілий тип, який називається integer. Діапазон цього типу залежить від реалізації, але в будь-якому випадку він включає в себе діапазон від $-(2^{31}-1)$ до $2^{31}-1$.

1.7.2. Фізичний тип

Фізичний тип – числовий тип для представлення деякої фізичної величини, наприклад, маси, довжини, часу чи напруги. Визначення фізичного типу включає визначення основної одиниці виміру і, можливо, ряду вторинних одиниць, які є кратними до основної одиниці:

```
визначення_фізичного_типу ::=
        обмеження_діапазону
                units
                        оголошення_основної_одиниці
                        { оголошення_вторинної_одиниці}
                end units
оголошення_основної_одиниці ::= ідентифікатор ;
оголошення_вторинної_одиниці ::= ідентифікатор = фізичний_літерал ;
фізичний_літерал ::= [ абстрактний_літерал ] ім'я_одиниці
```

Приклади оголошень фізичного типу:

```
type length is range 0 to 1E9
        units
                um;
                mm = 1000 um;
                cm = 10 mm;
```

```
                m = 1000 mm;
                in = 25.4 mm;
                ft = 12 in;
                yd = 3 ft;
                rod = 198 in;
                chain = 22 yd;
                furlong = 10 chain;
        end units;
type resistance is range 0 to 1E8
        units
                ohms;
                kohms = 1000 ohms;
                Mohms = 1E6 ohms;
        end units;
type capacity is range 0 to 10000000
        units
                pf;
                nf = 1000 pf;
                uf = 1000 nf;
                mf = 1000 uf;
end units;
```

Наперед визначений фізичний тип time є важливим у VHDL, оскільки він часто використовується для затримок. Він визначений таким чином:

```
type time is range визначається реалізацією
        units
                fs;
                ps = 1000 fs;
                ns = 1000 ps;
                us = 1000 ns;
                ms = 1000 us;
                sec = 1000 ms;
                min = 60 sec;
                hr = 60 min;
        end units;
```

Щоб записати значення деякого фізичного типу, потрібно записати число, а після нього одиницю виміру. Наприклад:

```
1 hr    1 kohms         1200 ohms       23 mf
```

1.7.3. Тип з плаваючою точкою

Тип з плаваючою точкою являє собою дискретне наближення до множини дійсних чисел у вказаному діапазоні. Точність цього наближення не визначена стандартом мови VHDL, але вона повинна бути принаймні шість десяткових цифр.

визначення_типу_з_плаваючою_точкою := обмеження_діапазону

Деякі приклади:

```
type signal_level is range -10.00 to +10.00;
type probability is range 0.0 to 1.0;
```

У VHDL є напередвизначений тип з плаваючою точкою, який називається real. Діапазон цього типу визначається реалізацією, але він повинен бути принаймні від -10^{38} до $+10^{38}$.

1.7.4. Тип перерахування

Тип перерахування складається з впорядкованого набору ідентифікаторів чи символів. Ідентифікатори і символи в межах одного типу перерахування повинні бути різними, однак вони можуть бути використані в різних типах перерахування.

визначення_типу_перерахування ::=

(літерал_перерахування { , літерал_перерахування })

літерал_перерахування ::= ідентифікатор | символьний_літерал

Деякі приклади:

```
type logic_level is (unknown, low, undriven, high);
type alu_function is (disable, pass, add, subtract, multiply, divide);
type octal_digit is ('0', '1', '2', '3', '4', '5', '6', '7');
```

У VHDL є ряд напередвизначених типів перерахування:

```
type severity_level is (note, warning, error, failure);
type boolean is (false, true);
type bit is ('0', '1');
type character is (
NUL,    SOH,    STX,    ETX,    EOT,    ENQ,    ACK,    BEL,
BS,     HT,     LF,     VT,     FF,     CR,     SO,     SI,
DLE,    DC1,    DC2,    DC3,    DC4,    NAK,    SYN,    ETB,
CAN,    EM,     SUB,    ESC,    FSP,    GSP,    RSP,    USP,
' ',    '!',    '"',    '#',    '$',    '%',    '&',    ''',
'(',    ')',    '*',    '+',    ',',    '-',    '.',    '/',
'0',    '1',    '2',    '3',    '4',    '5',    '6',    '7',
'8',    '9',    ':',    ';',    '<',    '=',    '>',    '?',
'@',    'A',    'B',    'C',    'D',    'E',    'F',    'G',
'H',    'I',    'J',    'K',    'L',    'M',    'N',    'O',
'P',    'Q',    'R',    'S',    'T',    'U',    'V',    'W',
'X',    'Y',    'Z',    '[',    '\',    ']',    '^',    '_',
'`',    'a',    'b',    'c',    'd',    'e',    'f',    'g',
'h',    'i',    'j',    'k',    'l',    'm',    'n',    'o',
'p',    'q',    'r',    's',    't',    'u',    'v',    'w',
'x',    'y',    'z',    '{',    '|',    '}',    '~',    DEL);
```

Тип character є прикладом типу перерахування, який містить суміш ідентифікаторів та символів. Зверніть увагу, що

символи '0' та '1' використовуються в типах bit та character. Якому типу відповідають ці символи, визначається з контексту.

На практиці часто використовують типи std_ulogic та std_logic. Вони не є частиною стандарту VHDL, а описані в пакеті (див. розділ 3.2.3) std_logic_1164, тому перед їх використанням потрібно описати відповідний контекст (див. розділ 4.1):

```
library IEEE;
use IEEE.std_logic_1164.all
```

Тип std_ulogic є типом перерахування, який визначено таким чином:

```
type std_ulogic is
    (       'U',    -- неініціалізовано
            'X',    -- невідоме значення (сильне джерело сигналу)
            '0',    -- 0 (сильне джерело сигналу)
            '1',    -- 1 (сильне джерело сигналу)
            'Z',    -- високий імпеданс
            'W',    -- невідоме значення (слабе джерело сигналу)
            'L',    -- 0 (слабе джерело сигналу)
            'H',    -- 1 (слабе джерело сигналу)
            '-'     -- невизначене значення);
```

Тип std_logic має таку ж саму множину значень, але дозволяє розв'язання конфліктів (див. Розділ 5.1).

1.7.5. Підтипи

Використання *підтипу* дозволяє обмежити значення, які може приймати об'єкт чи його індекс (для масивів), підмножиною деякого типу. Таким чином, значення належить підтипу, якщо воно належить деякому типу та задовольняє обмеженням. Цей тип називають базовим. Можна створити підтип підтипу і т.д. Всі такі підтипи мають один базовий тип і відрізняються лише обмеженнями. Зауважте, що оголошення підтипу не створює новий тип.

```
оголошення_підтипу ::= subtype ідентифікатор is опис_підтипу ;
опис_підтипу ::= [ ім'я_функції_розв'язання_конфліктів ]
        позначення_типу [ обмеження ]
позначення_типу ::= ім'я_типу | ім'я_підтипу
обмеження ::= обмеження_діапазону | обмеження_індексу
```

Підтип може застосовуватися для обмеження дозволених значень скалярного типу. В такому випадку вказуються граничні значення, які може приймати об'єкт цього підтипу. Наприклад:

```
subtype pin_count is integer range 0 to 400;
subtype digits is character range '0' to '9';
```

Підтип може використовуватися для визначення границь індексів у індексних типах (див. наступний розділ). Наприклад:

```
subtype id is string(1 to 20);
subtype word is bit_vector(31 downto 0);
```

У VHDL є два напередвизначених числових підтипи:

```
subtype natural is integer range 0 to highest_integer;
subtype positive is integer range 1 to highest_integer;
```

Величина highest_integer визначається реалізацією, але вона не менша ніж 2^{31}-1.

Підтипи використовуються і з сигналами. Для них описаний вище синтаксис може бути доповнений оголошенням імені функції розв'язання конфліктів (див. розділ 5.1).

1.7.6. Індексний тип

Масиви в VHDL являють собою індексовану сукупність елементів, які мають один і той самий тип. Масиви можуть бути одномірні (мають один індекс) та багатомірні (мають декілька індексів). Крім того, *індексний тип* може бути обмеженим, якщо границі індексу (чи індексів) встановлюються, коли описується тип, чи необмеженим, якщо границі встановлюються згодом, коли описується об'єкт даних цього типу.

```
визначення_індексного_типу ::=
        визначення_необмеженого_індексного_типу
        | визначення_обмеженого_індексного_типу
визначення_необмеженого_індексного_типу ::=
        array ( визначення_підтипу_індексу
        { , визначення_підтипу_індексу } )
        of опис_підтипу_елементів
визначення_обмеженого_індексного_типу ::=
        array обмеження_індексу of опис_підтипу_елементів
визначення_підтипу_індексу ::= позначення_типу range <>
обмеження_індексу ::= ( дискретний_діапазон { , дискретний_діапазон } )
дискретний_діапазон ::= опис_дискретного_підтипу | діапазон
```

Деякі приклади оголошень обмежених індексних типів:

```
type word_32 is array (31 downto 0) of bit;
type memory is array (address) of word;
type transform is array (1 to 4, 1 to 4) of real;
type register_bank is array (byte range 0 to 132) of integer;
```

Оголошення необмеженого індексного типу:

```
type vector is array (integer range <>) of real;
```

Символ “<>” означає, що межі індексу будуть вказані пізніше, коли буде оголошуватися об’єкт даних цього типу. Наприклад, об’єкт оголошений як

```
vector(1 to 20)
```

є вектором з 20 елементів типу real.

В мові VHDL є два напередвизначених індексних типи, обидва необмежені. Вони визначені так:

```
type string is array (positive range <>) of character;
type bit_vector is array (natural range <>) of bit;
```

Тип bit_vector особливо корисний для опису двійниково-кодованого представлення величин при моделюванні цифрових систем.

До елементів об’єктів індексного типу можна звертатися за допомогою імені об’єкта, за яким у круглих дужках стоїть індекс. Наприклад, припустимо, що a та b – одно- та двомірний масиви, відповідно. Тоді за допомогою імен a(1) та b(1,1) можна звертатися до елементів цих масивів. Крім того, замість одного індексу можна використовувати діапазон індексів. Наприклад, ім’я a(8 **to** 15) позначає восьмиелементний масив, який є частиною масиву a, такий масив називають *відрізком (slice)* масиву a.

Іноді потрібно записати літерал індексного типу. Це можна зробити, якщо створити *агрегат (aggregate)*, який будується безпосередньо із значень констант.

Наприклад, індексний тип визначений таким чином:

```
type a is array (1 to 4) of character;
```

Припустимо, нам потрібно записати величину такого типу, яка містить елементи 'a', 'b', 'c', 'c' у вказаному порядку. Це можна зробити так:

```
('a', 'b', 'c', 'c')
```

Елементи записуються в порядку індексного діапазону, починаючи з лівої границі діапазону. Така відповідність значення та індексу елемента масиву називається позиційною. Також можна використовувати іменовану відповідність значень та індексів, при якій явно вказуються пари індекс-значення:

```
(1 => 'a', 3 => 'c', 4 => 'c', 2 => 'b')
```

Слід зауважити, що при цьому елементи можуть бути записані в довільному порядку. Позиційну і іменовану форми запису можна використовувати в одному агрегаті, причому всі елементи, для яких використовується позиційна відповідність, повинні стояти на початку агрегату. Також можна

використовувати слово **others** як останній індекс при іменованій відповідності. За допомогою цього слова вказуються значення всіх елементів, які не згадані явно. Наприклад, та ж сама величина могла бути описана таким чином:

```
('a', 'b', others => 'c')
```

Зауважте, що такі самі правила використовуються для іменованої відповідності формальних та фактичних параметрів при викликах підпрограм, створенні екземплярів компонентів та створені блоків, але замість індексів використовують імена формальних параметрів та імена портів.

1.7.7. Записи

У VHDL є основні засоби для роботи з *записами (record)*. Записи є сукупностями іменованих елементів, можливо, різних типів, які називаються полями. Синтаксис визначення *іменованого типу* такий:

```
визначення_іменованого_типу ::=
        record
                оголошення_елемента
                { оголошення_елемента }
        end record
оголошення_елемента ::= список_ідентифікаторів :
        визначення_підтипу_елемента ;
список_ідентифікаторів ::= ідентифікатор { , ідентифікатор )
визначення_підтипу_елемента ::= опис_підтипу
```

Приклад оголошення іменованого типу:

```
type instruction is
        record
                op_code : processor_op;
                address_mode : mode;
                operand1, operand2: integer range 0 to 15;
        end record;
```

Для того, щоб звернутися до деякого поля запису, за іменем запису ставиться крапка, а потім – ім’я поля . Наприклад, якщо r – ім’я деякого запису, який містить поле, яке називається f, то ім’я r.f відноситься до цього поля.

Щоб записати літеральні значення записів можна використовувати агрегати. Можливе використання позиційної і іменованої відповідності значень та полів. При створенні агрегатів записів застосовуються ті ж самі правила, як і для агрегатів масивів, але замість індексів використовуються імена полів.

1.7.8. Оголошення об'єктів даних

Об’єкт даних – це іменована сутність, яка асоційована з деяким типом. Перед використанням об’єкт даних потрібно оголосити у розділі оголошень. У VHDL є три класи об’єктів даних: *константи (constant)*, *змінні (variable)* та *сигнали (signal)*. В цьому розділі будуть розглянуті тільки перші два класи, а сигнали будуть описані у розділах 2.2.1 та 3.4. Використання констант і змінних подібне їхньому використанню в алгоритмічних мовах програмування.

Константа являю собою об’єкт, який ініціалізується деяким значенням при оголошенні, і згодом не може змінюватися.

оголошення_константи ::=

constant список_ідентифікаторів : опис_підтипу [:= вираз];

Оголошення константи без виразу ініціалізації створює відкладену константу. Таке оголошення можна використовувати тільки у розділі оголошень пакетів (див. розділ 3.2.3). В такому випадку значення константи повинно бути вказано в тілі відповідного пакета. Деякі приклади:

```
constant e : real := 2.71828;
constant delay : Time := 5 ns;
constant max_size : natural;
```

Змінна – об’єкт даних, значення якого може змінюватися оператором присвоювання після того, як він створений. Перед використанням змінну потрібно оголосити:

оголошення_змінної ::=

variable список_ідентифікаторів : опис_підтипу [:= вираз] ;

Якщо в оголошенні змінної присутній ініціалізуючий вираз, то при оголошенні змінної він обраховується і присвоюється їй. Якщо цей вираз відсутній, то змінна ініціалізується значенням по замовчуванню. Значення по замовчуванню для скалярних типів – це крайнє ліве значення типу, тобто те, яке є першим у списку типу перерахування, найменшим у зростаючому діапазоні, чи найбільшим у спадаючому діапазоні. Якщо змінна має складений тип, то значенням по замовчуванню буде агрегат значень по замовчуванню кожного поля. Ці ж правила справедливі і для ініціалізації сигналів.

Деякі приклади оголошень змінних:

```
variable count : natural := 0;
```

variable trace : trace_array;
Якщо припустити, що тип trace_array – масив елементів типу boolean, тоді початкове значення змінної trace – масив з восьми елементів, які мають значення false.

Для існуючого об'єкта можна створити додаткову назву цього об'єкта чи його частини. Це робиться за допомогою оголошення *псевдоніма (alias)*:
оголошення_псевдоніма ::= **alias** ідентифікатор : опис_підтипу **is** ім'я;

Псевдонім інтерпретується як додаткове ім'я об'єкта чи його частини, яка вказана в оголошенні псевдоніма. Наприклад:
variable instr : bit_vector(31 **downto** 0);
alias op_code : bit_vector(7 **downto** 0) **is** instr(31 **downto** 24);
В цьому прикладі ім'я op_code є назвою восьми лівих бітів змінної instr.

1.7.9. Атрибути

Типи і об'єкти даних мають додаткову інформацію, яка називається їх *атрибутами (attribute)*. У VHDL існує ряд стандартних напередвизначених атрибутів. Деякі атрибути типів та сигналів розглянуті в цьому розділі. Для використання атрибута після імені об'єкта чи типу потрібно поставити символ ' та ім'я атрибута. Наприклад, вираз
thing'attr
повертає значення атрибута attr об'єкта thing.

Розглянемо напередвизначені атрибути, які можна використовувати для будь-якого скалярного типу чи його підтипу T:

Атрибут	Результат
T'left	Ліва границя T
T'right	Права границя T
T'low	Нижня границя T
T'high	Верхня границя T

Для зростаючого діапазону T'left = T'low та T'right = T'high. Для спадаючого діапазону T'left = T'high та T'right = T'low.

Для будь-якого дискретного чи фізичного типу або їх підтипу T можна використовувати такі атрибути:

Атрибут	Результат
T'pos(X)	Номер позиції X в T
T'val(N)	Елемент T в позиції з номером N
T'leftof(X)	Елемент T в позиції зліва від позиції X

T'rightof(X)	Елемент T в позиції справа від позиції X
T'pred(X)	Елемент T в позиції з номером на один меншим, ніж номер позиції X
T'succ(X)	Елемент T в позиції з номером на один більшим, ніж номер позиці X

В цих визначеннях X – довільний елемент типу T, N – ціле число,

Для зростаючого діапазону T'leftof (X) = T'pred (X), і T'rightof (X) = T'succ (X). Для спадаючого діапазону T'leftof (X) = T'succ (X), і T'rightof (X) = T'pred (X).

Для будь-якого індексного типу A чи об’єкта такого типу та натурального числа N, яке менше кількості розмінностей A, можуть використовуватися такі атрибути:

Атрибут	Результат
A'left(N)	Ліва границя діапазону N-го індексу масиву A
A'right(N)	Права границя діапазону N-го індексу масиву A
A'low(N)	Нижня границя діапазону N-го індексу масиву A
A'high(N)	Верхня границя діапазону N-го індексу масиву A
A'range(N)	Діапазон N-го індексу масиву A
A'reverse_range(N)	Діапазон N-го індексу масиву A в оберненому порядку
A'length(N)	Довжина діапазону N-го індексу масиву A

Для сигналу S, окрім вказаних вище, можуть використовуватися додаткові атрибути:

S'last_value	попереднє значення, тобто те, яке мав сигнал S перед останньою зміною
S'delayed(D)	значення, яке мав сигнал S D одиниць часу назад
S'stable(D)	атрибут типу boolean, який має значення true, якщо сигнал S не змінювався на протязі D останніх одиниць часу. Якщо D=0, то атрибут можна записати так: S'stable.
S'event	атрибут типу boolean, який має значення true, якщо сигнал S змінився на цьому кроці моделювання.

S'quiet(D) атрибут типу boolean, який має значення true, якщо на протязі D останніх одиниць часу не було виконано жодної трансакції сигналу S.

Слід зауважити, що тут перераховані далеко не всі атрибути, які існують в мові VHDL. Повний їх список наведено у відповідних стандартах.

1.8. Вирази та операції

Вирази в VHDL подібні виразам в мовах програмування. Вираз – це формула, яка складається з первинних виразів та операції. Первинними виразами являються назви об'єктів даних, виклики функції та вирази у в дужках. Далі наведені операції у порядку зменшення їх пріоритету:

Найвищий пріоритет:	******	**abs**	**not**			
	*****	/	**mod**	**rem**		
	знак+		знак–			
	+	–	&			
	=	/=	<	<=	>	>=
Найнижчий пріоритет:	**and**	**or**	**nand**	**nor**	**xor**	

Логічні операції **and**, **or**, **nand**, **nor**, **xor**, **not** працюють з аргументами типу bit та boolean, а також з одномірними масивами таких типів. Якщо їх операндами є масиви, то операція застосовується для відповідних елементів масивів, а результат буде масивом тієї ж самої розмірності, як і операнди.

Операції **and**, **or**, **nand**, **nor** скорочені, тобто вони обчислюють правий операнд, тільки якщо лівий операнд не визначає результат. Наприклад, операції **and** та **nand** обраховують правий операнд, тільки якщо лівий операнд є '1' чи true.

Вказані вище логічні операції також визначені і для операндів типів std_logic та std_ulogic. Наприклад, операція **and** описується такою таблицею істинності:

	'U'	'X'	'0'	'1'	'Z'	W	'L'	'H'	'-'
'U'	**'U'**	**'U'**	**'0'**	**'U'**	**'U'**	**'U'**	**'0'**	**'U'**	**'U'**
'X'	**'U'**	**'U'**	**'0'**	**'X'**	**'X'**	**'X'**	**'0'**	**'X'**	**'X'**
'0'	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**
'1'	**'U'**	**'X'**	**'0'**	**'1'**	**'X'**	**'X'**	**'0'**	**'1'**	**'X'**
'Z'	**'U'**	**'X'**	**'0'**	**'X'**	**'X'**	**'X'**	**'0'**	**'X'**	**'X'**
W	**'U'**	**'X'**	**'0'**	**'X'**	**'X'**	**'X'**	**'0'**	**'X'**	**'X'**
'L'	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**	**'0'**
'H'	**'U'**	**'X'**	**'0'**	**'1'**	**'X'**	**'X'**	**'0'**	**'1'**	**'X'**

'-'	**'U'**	**'X'**	**'0'**	**'X'**	**'X'**	**'X'**	**'0'**	**'X'**	**'X'**

В першому рядку та стовпчику приведені значення операндів, а жирним шрифтом у таблиці на перетині відповідного рядка та стовпчика наведено результат. Операції **or** з операндами типів std_logic та std_ulogic відповідає така таблиця істинності:

	'U'	'X'	'0'	'1'	'Z'	W	'L'	'H'	'-'
'U'	**'U'**	**'U'**	**'U'**	**'1'**	**'U'**	**'U'**	**'U'**	**'1'**	**'U'**
'X'	**'U'**	**'X'**	**'X'**	**'1'**	**'X'**	**'X'**	**'X'**	**'1'**	**'X'**
'0'	**'U'**	**'X'**	**'0'**	**'1'**	**'X'**	**'X'**	**'0'**	**'1'**	**'X'**
'1'	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**
'Z'	**'U'**	**'X'**	**'X'**	**'1'**	**'X'**	**'X'**	**'X'**	**'1'**	**'X'**
W	**'U'**	**'X'**	**'X'**	**'1'**	**'X'**	**'X'**	**'X'**	**'1'**	**'X'**
'L'	**'U'**	**'X'**	**'0'**	**'1'**	**'X'**	**'X'**	**'0'**	**'1'**	**'X'**
'H'	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**
'-'	**'U'**	**'X'**	**'X'**	**'1'**	**'X'**	**'X'**	**'X'**	**'1'**	**'X'**

Операції відношення =, /=, <, <=, > та >= повертають результат типу boolean. Обидва операнди цих операцій повинні мати один тип. Операції рівності (= та /=) можуть мати операнди будь-якого типу. Для записів два значення рівні, якщо всі відповідні поля рівні. Операнди всіх інших операцій відношення повинні бути або скалярного типу, або бути одномірними масивами з елементами дискретних типів.

Операції знаку (+ і -), додавання (+) та віднімання (-) мають звичайний зміст на числових операндах. Операція конкатенації (&) об'єднує два одномірних масиви і створює новий масив, на початку якого містяться елементи правого операнду, а за ними – елементи лівого операнду. За допомогою цієї операції можна також приєднати новий елемент до масиву чи об'єднати декілька окремих елементів, щоб сформувати масив. Операція конкатенації часто використовується з рядками.

Операції множення (*) і ділення (/) працюють з цілими типами, фізичними типами та типами з плаваючою точкою. Операції модуля (**mod**) та залишку (**rem**) працюють тільки з цілими типами. Операція абсолютного значення (**abs**) може використовуватися з будь-яким числовим типом. Нарешті, у операції зведення у ступінь (**) лівий операнд може бути цілим або з плаваючою точкою, а правий операнд повинен бути цілим. Від'ємний правий операнд допускається, тільки якщо лівий операнд - число з плаваючою точкою.

2. Структурний опис цифрових систем на мові VHDL

У главі 1 були оглянуті основні засоби мови VHDL, які застосовуються для опису структури цифрової системи. В цій главі буде детально розглянуто створення структурних описів на мові VHDL.

2.1. Оголошення об'єкта

Цифрова система звичайно розробляється як ієрархічна сукупність модулів. Кожен модуль має набір портів, які складають його інтерфейс до оточення. У VHDL об’єкт – такий модуль, він може використовуватися як компонент проекту, чи може бути модулем верхнього рівня. Синтаксис оголошення об’єкта такий:

оголошення_об’єкта ::=
    entity ідентифікатор **is**
        заголовок_об’єкта
        розділ_оголошень_об’єкта
    [**begin**
        операторна_частина_оголошення_об’єкта]
    end [просте_ім’я_*об’єкта*] ;
заголовок_об’єкта ::=
    [розділ_*формальних*_параметрів_настроювання]
    [розділ_*формальних*_портів]
розділ_параметрів_настроювання ::=
    generic (список_параметрів_настроювання) ;
список_параметрів_настроювання ::=
    інтерфейсний_список_*параметрів_настроювання*
розділ_портів ::= **port** (список_портів) ;
список_портів ::= інтерфейсний_список_*портів*
розділ_оголошень_об’єкта ::= { елемент_оголошення_об’єкта }

У розділі оголошень об’єкта можуть оголошуватися елементи (компоненти, типи, константи, сигнали), які будуть використовуватися у реалізації цього об’єкта. Частіше вони оголошуються саме у реалізації (в архітектурному тілі об’єкта), тому тут вони не будуть розглядатися. Також в оголошенні об’єкта можуть бути розташовані необов’язкові оператори, які призначені для контролю за роботою об’єкта. Вони будуть розглянуті в розділі 5.4.

Заголовок об’єкта – найбільш важлива частина оголошення об’єкта. Він включає специфікацію параметрів настроювання, які використовуються, щоб керувати структурою і поведінкою об’єкта, і портів, через які інформація передається в об’єкт та з нього.

Параметри настроювання та порти визначаються з використанням інтерфейсного списку, подібного інтерфейсному списку підпрограми (див. розділ 3.2.1):

```
інтерфейсний_список ::= елемент_інтерфейсу { ; елемент_інтерфейсу }
елемент_інтерфейсу ::= оголошення_інтерфейсу
оголошення_інтерфейсу::=
        оголошення_інтерфейсних_констант
        | оголошення_інтерфейсних_сигналів
        | оголошення_інтерфейсних_змінних
оголошення_інтерфейсних_констант ::=
        [ constant ] список_ідентифікаторів : [ in ] опис_підтипу
        [ := статичний_вираз ]
оголошення_інтерфейсних_змінних ::=
        [ variable ] список_ідентифікаторів : [ режим ] опис_підтипу
        [ := статичний_вираз ]
оголошення_інтерфейсних_сигналів ::=
        [ signal ] список_ідентифікаторів : [ режим ] опис_підтипу [ bus ]
        [ := статичний_вираз ]
```

Опис підтипу був розглянутий в розділі 1.7.5.

Всі параметри настроювання повинні оголошуватися як інтерфейсні константи. Порти об’єкта визначаються з використанням інтерфейсного списку, подібного інтерфейсному списку параметрів настроювання, але всі його елементи повинні бути сигналами. Змінні не можуть використовуватися у інтерфейсах об’єктів.

Слово **signal** в інтерфейсному списку сигналів може бути опущено. Слово **bus** використовується, якщо можлива ситуація, коли всі драйвери сигналу від’єднанні (див. розділ 5.5 та 5.6). Як і параметри настроювання, фактичні сигнали, які будуть з’єднані з портами, визначаються, коли об’єкт використовується в якості екземпляра компонента у проекті.

Режим (mode) інтерфейсного сигналу вказує можливість його зчитування та зміни у об’єкті. Якщо режим сигналу **in**, то він може бути тільки зчитаний в об’єкті, змінити його неможливо. Сигнал, оголошений з режимом **out,** може бути змінений в

об’єкті, але дізнатися про його значення в об’єкті неможливо. Сигнал **inout** може бути зчитаний та змінений в об’єкті.

Розглянемо декілька прикладів оголошень об’єктів:

```
entity processor is
        generic (max_clock_freq : frequency := 30 MHz);
        port (  clock : in bit;
                address : out integer;
                data : inout word_32;
                control : out proc_control;
                ready : in bit);
end processor;
```

У цьому прикладі параметр настроювання max_clock_freq визначає часові параметри об’єкта. Код, який описує поводження об’єкта, буде використовувати цю величину, щоб визначити затримки змін сигналів.

Наступний приклад показує, як параметри настроювання можуть використовуватися, щоб визначити клас об’єктів зі змінною структурою:

```
entity ROM is
        generic (width, depth : positive);
        port (  enable : in bit;
                address : in bit_vector(depth–1 downto 0);
                data : out bit_vector(width–1 downto 0) );
end ROM;
```

У цьому прикладі параметри настроювання визначають кількість інформаційних та адресних бітів у постійному запам’ятовуючому пристрої. Зверніть увагу на те, що в цьому прикладі параметри настроювання не мають значень по замовчуванню. Це означає, що коли об’єкт буде використовуватися як екземпляр компонента, то завжди повинні бути вказані фактичні величини цих параметрів.

Нарешті, приклад оголошення об’єкта без параметрів настроювання та без портів:

```
entity test_bench is
end test_bench;
```

Хоча такий об’єкт спочатку здається некорисним, такі об’єкти використовуються як іспитові стенди при тестуванні інших об’єктів (див. розділ 4.3). Об’єкт верхнього рівня тестового стенду складається зі з’єднаних між собою генератора тестових сигналів та об’єкта, робота якого досліджується. Значення сигналів можуть бути простежені системою моделювання чи перевірені прямо генератором тестових сигналів. Ніякі зовнішні

підключення до іспитового стенда непотрібні, отже він не має портів.

2.2. Архітектурне тіло об'єкта

Після того, як об'єкт оголошений, його реалізація повинна бути описана в архітектурному тілі. Одному об'єкту може відповідати декілька архітектурних тіл. Кожне архітектурне тіло являє собою різне представлення об'єкта. Наприклад, одне архітектурне тіло може описувати поведінку, використовуючи алгоритмічні засоби, розглянуті у главі 3, а інші можуть описувати структуру об'єкта як ієрархічну сукупність з'єднаних між собою компонентів. У цьому розділі ми розглянемо тільки структурні описи.

Синтаксис опису архітектурного тіла такий:

архітектурне_тіло ::=
    architecture ідентифікатор **of** назва_*об'єкта* **is**
        розділ_оголошень_архітектурного_тіла
    begin
        операторна_частина_архітектурного_тіла
    end [просте_ім'я_*архітектурного_тіла*] ;
розділ_оголошень_архітектурного_тіла ::=
    { елемент_оголошення_блоку }
операторна_частина_архітектурного_тіла ::= { паралельний_оператор }
елемент_оголошення_блоку ::=
    оголошення_підпрограми
    | тіло_підпрограми
    | оголошення_типу
    | оголошення_підтипу
    | оголошення_константи
    | оголошення_сигналу
    | оголошення_компонента
    | оголошення_псевдоніма
    | оголошення_конфігурації
    | специфікатор_використання
паралельний_оператор ::=
    оператор_блоку
    | оператор_створення_екземпляра_компонента
    | оператор_процесу
    | оператор_паралельного_присвоювання
    | оператор_виклику_присвоювання
    | оператор_паралельного_виклику_процедури
    | оператор_паралельної_генерації
    | оператор_паралельного_повідомлення

Оголошення в архітектурному тілі визначають елементи, які будуть використовуватися, щоб створити опис об'єкта. Зокрема, тут можуть бути оголошені сигнали і компоненти, які потім використовуються, щоб створити структурний опис. Оголошення констант, типів, підтипів, псевдонімів були розглянуті в главі 1, а оголошення підпрограм буде розглянуте в главі 3.

Операторна частина архітектурного тіла складається *тільки* з паралельних операторів, таких, як оператори створення екземплярів компонентів, процесів, блоків, паралельного присвоювання та паралельного виклику підпрограм (які еквівалентні процесам). Вони будуть розглянуті в цій главі та главах 3 та 5.

2.2.1. Оголошення сигналів

Сигнали використовуються, щоб передавати інформацію між об'єктами (екземплярами компонентів, процесами, блоками), які розташовані в архітектурному тілі. Перед використанням сигнали потрібно оголосити:

оголошення_сигналу ::=
    signal список_ідентифікаторів : опис_підтипу
    [клас_сигналу]
    [:= вираз] ;
клас_сигналу ::= **register | bus**

Використання *класу (kind)* сигналу **register** та **bus** буде розглянуто у розділах 5.5 та 5.6. Вираз в оголошенні може використовуватися, щоб протягом стадії ініціалізації моделювання присвоїти сигналу початкове значення . Якщо він відсутній, то сигналу буде присвоєно початкове значення по замовчуванню (див. розділ 1.7.8).

Слід зауважити, що порти об'єкта в межах цього об'єкта трактуються точно так, як і сигнали.

2.2.2. Блоки

Субмодулі в архітектурному тілі можуть бути описані як блоки. *Блок (block)* - це одиниця модульної структури з його власним інтерфейсом, яка з'єднана з іншими блоками чи портами сигналами. Блок складається з інтерфейсної частини, локальних оголошень та групи паралельних операторів. Саме архітектурне

тіло є також блоком. Сигнали, константи та інші одиниці, які оголошені у блоці, не видимі за його межами.

оператор_блоку ::=
    мітка_*блоку* :
        block [(*захисний*_вираз)]
        заголовок_блоку
        розділ_оголошень_блоку
    begin
        операторна_частина_блоку
    end block [мітка_*блоку*] ;
заголовок_блоку ::=
    [розділ_параметрів_настроювання
    [розділ_співставлення_параметрів_настроювання ;]]
    [розділ_портів
    [розділ_співставлення портів ;]]
розділ_співставлення_параметрів_настроювання ::=**generic map**
    (список_співставлення_*параметрів_настроювання*)
розділ_співставлення портів ::= **port map**
    (список_співставлення_*портів*)
розділ_оголошень_блоку ::=
    { елемент_оголошення_блоку }

Захисний вираз блоку буде розглянутий у розділі 5.6. Заголовок блоку визначає інтерфейс блоку, подібно тому, як заголовок об'єкта визначає його інтерфейс. Список співставлення параметрів настроювання призначений для співставлення параметрів настроювання, які описані у зовнішній архітектурі чи у зовнішньому блоці, та параметрів, які оголошені і будуть використовуватися у блоці. Список співставлення портів визначає, які сигнали чи порти зовнішнього блоку чи архітектурного тіла з'єднані з портами блоку. Ці співставлення можуть бути позиційними та іменованими.

Слід відзначити, що у блоці можуть також міститися оператори блоків, так що проект може складатися з багаторівневої ієрархії блоків. Таким чином, блоки дозволяють відобразити ієрархічну структуру реальної системи.

Як приклад, розглянемо структурне архітектурне тіло об'єкта processor, який був оголошений у розділі 2.1. Якщо у процесорі виділено блок керування (control_unit) та блок тракту даних (data_path), то ми можемо записати архітектурне тіло, яке складається з двох зв'язаних блоків:

architecture block_structure **of** processor **is**
    type data_path_control is … ;
    signal internal_control : data_path_control;

```
begin
        control_unit : block
                        --розділ оголошень блоку control_unit
                        port (  clk : in bit;
                                bus_control : out proc_control;
                                bus_ready : in bit;
                                control : out data_path_control);
                        port map (clk => clock, bus_control => control,
                                bus_ready => ready,
                                control => internal_control);
                begin
                        оператори блоку control_unit
                end block control_unit;
        data_path : block
                        --розділ оголошень блоку data_path
                        port (  address : out integer;
                                data : inout word_32;
                                control : in data_path_control);
                        port map (address => address, data => data,
                                control => internal_control);
                begin
                        оператори блоку data_path
                end block data_path;
end block_structure;
```

Блок керування (control_unit) має порти clk, bus_control та bus_ready, які з'єднані з портами об'єкта processor, відповідно clock, control та ready. Він також має зв'язок через сигнал internal_control, який оголошений в архітектурному тілі, з блоком тракту даних (data_path). Цей сигнал з'єднаний з портами control обох блоків. Порти address та data блоку тракту даних (data_path) з'єднані з однойменними портами об'єкта processor. Перевага такої розбивки на блоки полягає в тому, що кожен із блоків може бути розроблений незалежно, а зв'язок між ними відбувається тільки через чітко описані інтерфейси.

2.2.3. Оголошення компонентів

В архітектурному тілі можуть також використовуватися інші об'єкти та їх архітектурні тіла. Щоб робити це, в розділі оголошень архітектури потрібно вказати відповідні компоненти. Компоненти подібні панелям для мікросхем, в яких можуть бути розташовані різні мікросхеми, але вони повинні мати зазначену кількість та тип виводів. Тобто, оголошення компонента задає лише інтерфейс, якому повинен задовольняти інтерфейс об'єкта,

який буде використовуватися в проекті. Синтаксис оголошення компонента такий:

оголошення_компонента ::=

component ідентифікатор

[розділ_*локальних*_параметрів_настроювання]

[розділ_*локальних*_портів]

end component

Деякі приклади оголошень компонентів:

```
component nand3
        generic (Tpd : Time := 1 ns);
        port (a, b, c : in logic_level;
                y : out logic_level);
end component;

component read_only_memory
        generic (data_bits, addr_bits : positive);
        port (en : in bit;
                addr : in bit_vector(depth–1 downto 0);
                data : out bit_vector(width–1 downto 0) );
end component;
```

У першому прикладі оголошується трививідний елемент nand3 з параметром настроювання, який визначає затримку сигналу при його проходженні через елемент. Потім можуть бути створені декілька екземплярів цього компонента з, можливо, різними затримками. У другому прикладі декларується компонент постійного запам'ятовуючого пристрою, розрядність адресної шини та шини даних якого визначається параметром настроювання.

2.2.4. Створення екземплярів компонентів

В архітектурному тілі можна створити екземпляри компонентів, які оголошені в ньому:

оператор_створення_екземпляра_компонента ::=

мітка_*екземпляра* : назва_*компонента*

[розділ_співставлення_параметрів_настроювання]

[розділ_співставлення_портів] ;

Такий оператор вказує, що архітектурне тіло містить копію об'єкта нижчого рівня ієрархії та його архітектурного тіла. В цьому операторі вказуються фактичні значення параметрів настроювання та імена сигналів та портів, які під'єднані до цього екземпляра компонента. Співставлення портів та параметрів

настроювання екземплярів компонентів та блоків повністю аналогічні.

Екземпляри компонентів, які оголошені в попередньому розділі, могли бути створені таким чином:

```
enable_gate: nand3
        port map (en1, en2, int_req, interrupt);
parameter_rom: read_only_memory
        generic map (data_bits => 16, addr_bits => 8)
        port map (en => rom_sel, data => param, addr => a(7 downto 0);
```

У першому прикладі значення параметра настроювання не вказано, так що для параметра Tpd буде використане значення по замовчуванню. В цьому прикладі використано позиційне співставлення імен сигналів та портів компонента. В другому прикладі вказані розрядності адресної шини та шини даних. У цьому прикладі використана іменована відповідність сигналів та портів компонента. Зауважте, що сигнал, який зв'язаний з портом addr – відрізок сигналу типу масив. Цей приклад ілюструє загальну практику використання шин, коли порт, який є масивом, може бути з'єднаний з частиною сигналу, який є масивом більшої розмірності.

Часто архітектурне тіло та об'єкти, які потрібно використовувати в ньому в якості екземплярів компонентів, описані в одному файлі. В такому випадку для використання такого об'єкта достатньо, щоб його ім'я та інтерфейс співпадали і іменем та інтерфейсом оголошеного компонента. Якщо об'єкт, який потрібно використати в якості екземпляра компонента, розташований в бібліотеці, або він має ім'я, які відрізняється від імені оголошеного компонента, то потрібно підключити відповідну бібліотеку та/чи описати конфігурацію (див. розділи 4.1 та 4.2).

Слід відзначити, що в стандарті VHDL 1993 року були введені два нових способи створення ієрархічної структури проекту. Це створення екземпляра об'єкта та екземпляра конфігурації в архітектурному тілі. В цих випадках попередня оголошення не потрібна. Ці можливості будуть розглянуті в розділі 5.2.

2.2.5. Оператор паралельної генерації

VHDL має додатковий паралельний оператор, який може використовуватися в архітектурних тілах, щоб створити

регулярні структури, такі, як масиви блоків, екземплярів компонентів чи процесів. Це оператор паралельної генерації, який містить в собі будь-які інші паралельні оператори, в тому числі і інші оператори паралельної генерації. Його синтаксис такий:

```
оператор_паралельної_генерації ::=
        мітка_генерації :
                схема_генерації generate
                        { паралельні_оператори }
                end generate [ мітка_генерації ] ;
схема_генерації ::=
        for специфікація_параметра_генерації
        | if умова
специфікація_параметра ::=
        ідентифікатор in дискретний_діапазон
```

Схема генерації описує регулярну структуру моделі. Специфікація параметра в цьому операторі співпадає з специфікацією параметра циклу **for**, який описано в розділі 3.1.4. Умовна схема генерації звичайно використовується, щоб обробити виняткові випадки в межах регулярної структури, наприклад такі, які відбуваються на її границях. Це проілюстровано наступним прикладом. Припустимо, що ми хочемо описати структуру багаторозрядного суматора. Він складається з екземплярів повного суматора (adder), за винятком найменшого значущого біта, де використовується напівсуматор (half_adder). Цей суматор можна описати таким чином:

```
adder_width : for i in 0 to width-1 generate
        least_bit : if i = 0 generate
                ls_cell : half_adder port map (a(0), b(0), sum(0), c_in(1));
        end generate lsbit;
        middle_bit : if i > 0 and i < width-1 generate
                middle_cell : adder port map (  a(i), b(i), c_in(i),
                                                sum(i), c_in(i+1));
        end generate middle_bit;
        most_bit : if i = width-1 generate
                ms_cell : adder port map (      a(i), b(i), c_in(i),
                                                sum(i), carry);
        end generate ms_bit;
end generate adder_width;
```

Зовнішній оператор генерації перебирає всі значення i в інтервалі від 0 до width-1. Для найменшого значущого біта (i=0) генерується екземпляр напівсуматора (half_adder). Його входи з'єднані з найменшими значущими бітами вхідних сигналів суматора a та b,

вихід з’єднаний з найменшим значущим бітом суми, а вихід переносу з’єднаний зі входом переносу наступного розряду. Для проміжних бітів генеруються екземпляри повного суматора adder, а їх входи та виходи з’єднані подібно першому розряду. Для старшого значущого біта (i=width-1) також генерується повний суматор, але його вихід переносу з’єднаний з виходом переносу суматора (carry).

3. Поведінковий опис систем в VHDL

Як було вказано в розділі 1.2, робота модуля може бути описана з використанням засобів, які притаманні алгоритмічним мовам програмування. Вони розглянуті у цій главі. Алгоритмічні засоби VHDL були розроблені на основі мови Ада. Завдяки цьому, не потрібно багато часу та зусиль для того, щоб познайомитися з ними. Однак, слід зауважити, що алгоритмічні засоби VHDL не настільки всебічні, як Ади, хоча вони адекватні для більшості цілей моделювання.

У цій главі будуть розглянуті послідовні оператори мови VHDL та два паралельні оператори, які застосовуються для створення поведінкового опису системи – оператор процесу та паралельного присвоювання сигналу.

3.1. Послідовні оператори

В мові VHDL існує ряд засобів для зміни стану об'єктів даних та керування потоком виконання поведінкового опису об'єкта. Вони будуть розглянуті в цьому розділі.

3.1.1. Оператор присвоювання змінній

У VHDL, як і в алгоритмічних мовах програмування, змінній можна присвоїти нове значення за допомогою оператора присвоювання.

оператор_присвоювання_змінній ::= адресат := вираз ;
    адресат ::= ім'я_змінної | агрегат

Якщо адресат є ім'ям змінної, то її значення заміняється новим значенням, яке задається виразом. Об'єкт, якому присвоюється нове значення, і вираз повинні мати один базовий тип. Якщо зліва від знаку := стоїть агрегат, то його елементи повинні бути іменами об'єктів даних, а значення виразу справа від := повинно бути того ж самого типу, як агрегат. Спочатку визначаються всі назви в агрегаті, потім обчислюється вираз справа від оператора присвоювання і, нарешті, значення компонент виразу присвоюються об'єктам, які входять в агрегат. Це, по суті, паралельне присвоювання.

Не слід плутати оператори присвоювання значення змінній та сигналу. Останній буде розглянуто в розділі 3.4.

3.1.2. Умовний оператор

Умовний оператор дозволяє вибрати групу операторів, яка буде виконуватися в залежності від однієї чи декількох умов.

```
умовний_оператор ::=
        if умова then
                послідовність_операторів
        { elsif condition then
                послідовність_операторів }
        [ else
                послідовність_операторів ]
        end if;
```

Умови повинні мати значення типу boolean. Вони обчислюються послідовно, поки не зустрінеться така, значення якої буде істина (true). Після цього виконується відповідна послідовність операторів. Якщо жодна умова не має значення true, виконується конструкція else, якщо вона присутня.

Приклади використання умовного оператора:

```
if (buf = null) or (buf'length = 0) then
        char := '*';
else
        read(buf, char);
end if;

if reset='1' then
        q <= '0';
elsif (ck= '1' and ck'event) then
        q <= d;
end if;
```

3.1.3. Оператор вибору

Оператор вибору case дозволяє вибрати групу операторів, яка буде виконуватися в залежності від значення виразу вибору. Синтаксис оператора case такий:

```
оператор_вибору ::=
        case вираз is
                альтернатива_оператора_вибору
                { альтернатива_оператора_вибору}
        end case ;
альтернатива_оператора_вибору ::=
        when список_виборів =>
                послідовність операторів
список_виборів ::= вибір { | вибір }
вибір ::=
        простий_вираз
```

| дискретний_діапазон
| просте_ім'я_*елементу*
| **others**

Значення виразу вибору повинно або бути дискретного типу, або бути одномірним масивом символів. Варіант, в список виборів якого входить значення виразу вибору, буде обраним і буде виконуватися тільки його послідовність операторів. Зауважте, що всі списки виборів повинні бути різними, тобто ніяке значення не може зустрічатися у декількох списках. Крім того, всі значення повинні бути представлені в списках виборів. В операторі case може використовуватися спеціальний варіант вибору others. Якщо значення виразу не зустрілося ні в якому списку виборів, виконується варіант others, який повинен бути останнім у операторі. Якщо вираз вибору повертає масив, то у списках виборів повинні стояти рядки.

Зауважте, що всі вирази у списках виборів повинні бути локально статичними, тобто їх значення повинно визначатися на стадії аналізу чи компонування проекту (див. розділ 4.1), а не при моделюванні.

Деякі приклади використання оператора вибору:

```
case element_colour of
        when red =>
                statements for red;
        when green | blue =>
                statements for green or blue;
        when orange to turquoise =>
                statements for these colours;
end case;

case opcode of
        when "00" => perform_add;
        when "01" => perform_subtract;
        when others => signal_illegal_opcode;
end case;

case val is
        when '0' => out1 <= '0' ;
        when '1' => out1 <= '1' ;
        when others => out1 <= val;
end case;
```

3.1.4. Оператор циклу

В VHDL існує оператор циклу, який може використовуватися для створення циклів while, until та for.

```
оператор_циклу ::=
        [ мітка_циклу : ]
                [ схема_ітерацій ] loop
                        послідовність_операторів
                end loop [ мітка_циклу ] ;
схема_ітерацій ::=
        while умова
        | for специфікація_параметра_циклу
специфікація_параметра ::=
        ідентифікатор in дискретний_діапазон
```

Якщо схема ітерацій відсутня, то цикл буде повторюватися нескінченно (якщо в ньому відсутній оператор виходу, див. далі). Приклад такого циклу:

```
loop
        do_something;
end loop;
```

Схема ітерацій дозволяє вказати умову, яка буде перевірятися перед кожним виконанням послідовності операторів. Якщо ця умова істинна, то послідовність операторів виконується, а якщо помилкова, то робота оператора циклу закінчується. Приклад:

```
while index < length and str(index) /= ' ' loop
        index := index + 1;
end loop;
```

Схема ітерацій **for** дозволяє виконати точно встановлену кількість ітерацій. В цій схемі ітерацій параметр циклу приймає по черзі всі значення з вказаного діапазону. Для кожного значення параметра циклу виконується послідовність операторів. В операторах, які створюють тіло циклу, параметр циклу трактується як константа, і він не може бути змінений оператором присвоювання. Параметр циклу не існує поза оператором циклу і його не потрібно оголошувати перед використанням. Наприклад:

```
for item in 1 to last_item loop
        table(item) := 0;
end loop;
```

Існує два додаткових оператора, які можуть використовуватися всередині циклу, щоб змінити основну схему ітерації. Оператор **next** перериває виконання поточної ітерації і

починає наступну. Оператор **exit** перериває виконання поточної ітерації і закінчує цикл.

оператор_наступної_ітерації ::= **next** [мітка_*циклу*] [**when** умова] ;
оператор_виходу ::= **exit** [мітка_*циклу*] [**when** умова] ;

Мітка циклу може застосовуватися, якщо ці оператори використовуються у вкладених циклах. Якщо мітка циклу відсутня, то ці оператори стосуються самого внутрішнього циклу, інакше вони застосовуються до вказаного циклу. Якщо після слова **when** присутня умова, але вона помилкова, ітерації продовжуються як звичайно. Деякі приклади:

```
for i in 1 to max_str_len loop
        a(i) := buf(i);
        exit when buf(i) = NUL;
end loop;
outer_loop : loop
        inner_loop : loop
                do_something;
                next outer_loop when temp = 0;
                do_something_else;
        end loop inner_loop;
end loop outer_loop;

for i in pos to l'right loop
        digit := digit_value(l(i));
        exit when (digit = not_digit) or (digit >= radix);
        result := result * radix + digit;
        pos := i + 1;
end loop;
```

3.1.5. Нульовий (порожній) оператор

Нульовий оператор нічого не робить. Його використовують, щоб явно показати, що в деяких випадках не потрібна ніяка дія. Найбільш часто він зустрічається в операторі **case**, коли потрібно перелічити всі можливі значення виразу вибору, але для деяких з них ніяка дія не потрібна. Наприклад:

```
case controller_command is
        when forward => engage_motor_forward;
        when reverse => engage_motor_reverse;
        when idle => null;
end case;
```

3.1.6. Оператор повідомлення

Оператор повідомлення використовується, щоб перевірити вказану умову та повідомити, якщо вона порушена. Його синтаксис такий:

оператор_повідомлення ::=
    assert умова
        [**report** вираз]
        [**severity** вираз];

Коли слово **report** присутнє, то, якщо умова *помилкова*, буде виведений вираз (в більшості випадків це просто рядок), який стоїть після слова **report.** Якщо слово **report** відсутнє, то виводиться стандартне повідомлення "Assertion violation". Також після слова **severity** можна вказати рівень серйозності помилки. Вираз, який стоїть після слова **severity**, повинен мати значення типу severity_level. Якщо він відсутній, то по замовчуванню рівень серйозності помилки error. Система моделювання може перервати моделювання, якщо рівень серйозності перебільшує деяке порогове значення. Величина цього порогу залежить від реалізації, але користувач може його змінити.

3.2. Підпрограми і пакети

Як і в мовах програмування, в VHDL можна використовувати підпрограми у формі функції та процедур. В VHDL також існують *пакети (package)*, які призначені для групування оголошень та підпрограм у єдиний модуль. Пакети являються механізмом абстракції даних та приховування інформації.

3.2.1. Процедури та функції

Синтаксис оголошення процедур та функції такий:

оголошення_підпрограми ::= специфікація_підпрограми ;
специфікація_підпрограми ::=
    procedure ідентифікатор [(список_формальних_параметрів)]
    | **function** ідентифікатор [(список_формальних_параметрів)]
    return позначення_типу

Оголошення підпрограми в цій формі просто називає підпрограму і визначає формальні параметри. Тіло підпрограми, яке визначає її дію, описується пізніше. Для функції оголошення також визначає тип результату, який повертається функцією. Ця форма оголошення підпрограми звичайно використовується в

пакеті (див. розділ 3.2.3), коли тіло підпрограми приводиться у його тілі, чи коли визначаються процедури, які викликають одна одну (взаємно рекурсивні процедури). Визначення формальних параметрів підпрограми є аналогічним визначенню портів та параметрів настроювання об'єктів:

список_формальних_параметрів ::= інтерфейсний_список_*параметрів*
інтерфейсний_список ::= елемент_інтерфейсу { ; елемент_інтерфейсу }
елемент_інтерфейсу ::= оголошення_інтерфейсу
оголошення_інтерфейсу::=
    оголошення_інтерфейсних_констант
    | оголошення_інтерфейсних_сигналів
    | оголошення_інтерфейсних_змінних
оголошення_інтерфейсних_констант ::=
    [**constant**] список_ідентифікаторів : [**in**] опис_підтипу
    [:= *статичний*_вираз]
оголошення_інтерфейсних_змінних ::=
    [**variable**] список_ідентифікаторів : [режим] опис_підтипу
    [:= *статичний*_вираз]
оголошення_інтерфейсних_сигналів ::=
    [**signal**] список_ідентифікаторів : [режим] опис_підтипу [**bus**]
    [:= *статичний*_вираз]

Формальні параметри підпрограм можуть бути константами, змінними та сигналами.

Розглянемо декілька прикладів. В першому прикладі оголошується процедура без параметрів:

```
procedure init;
```

В цьому прикладі оголошена процедура, яка називається init, чиє тіло буде описано згодом у пакеті. Виклик цієї процедури має такий вигляд:

```
init;
```

В наступному прикладі оголошується процедура з формальними параметрами:

```
procedure inc_acc(      variable acc : inout word_32;
                        constant inc : in integer := 0);
```

У процедурі inc_acc має два параметри, перший називається acc, а другий – inc. Формальний параметр acc є змінною, тобто в тілі процедури він обробляється як змінна і може стояти зліва в операторі присвоювання. Режим параметра acc – **inout**, це означає, що в тілі процедури змінна acc може як зчитуватися, так і змінюватися оператором присвоювання. Отжео, коли процедура викликається, фактичний параметр, який підставляється замість формального параметра acc, повинен бути змінною. Інші режими

параметрів підпрограми – **in**, який означає, що параметр може тільки читатися, і **out**, який означає, що параметр можна тільки змінити в тілі підпрограми. Якщо режим параметра **inout** чи **out**, то слово **variable** може бути опущено.

Другий формальний параметр, inc, є константою, тобто він обробляється як константа у тілі підпрограми і не може бути змінений. Якщо режим параметра **in**, то слово **constant** може бути опущено. Вираз після символу присвоювання (в цьому прикладі 0) – це вираз по замовчуванню, який використовується, якщо при виклику підпрограми не вказано ніякого фактичного параметра для inc.

Виклик підпрограми включає її ім'я та список фактичних параметрів, які будуть зв'язані з формальними параметрами. Відповідність формальних і фактичних параметрів у цьому списку може бути позиційна, іменована чи змішана, коли для деяких параметрів використовується позиційна відповідність, а для інших – іменована (як і при створенні агрегатів складених типів чи екземплярів компонентів). Якщо використовується позиційна відповідність, то при виклику підпрограми фактичні параметри повинні стояти в тому самому порядку, як і формальні. Наприклад:

```
inc_acc(counter, N);
```

inc_acc(counter); (тут для inc використовується значення по замовчуванню)

Якщо підпрограма викликається з використанням іменованої відповідності формальних та фактичних параметрів, пари формальний=>фактичний параметр можуть стояти у довільному порядку. Наприклад:

```
inc_acc(inc => N, reg => counter);
inc_acc(reg => counter);
```

В наступному прикладі оголошується функція:

```
function byte_to_int(byte : word_8) return integer;
```

Ця функція має один формальний параметр. Для функцій режим всіх формальних параметрів повинен бути **in**, і його можна явно не вказувати. Значення, яке повертає тіло цієї функції, повинне бути цілим числом.

Тіло підпрограми визначається таким чином:

```
тіло_підпрограми ::=
        специфікація_підпрограми is
                розділ_оголошень_підпрограми
```

begin
операторна_частина_підпрограми
end [позначення] ;
розділ_оголошень_підпрограми ::=
{ елемент_оголошення_підпрограми }
операторна_частина_підпрограми ::= { послідовний_оператор }
елемент_оголошення_підпрограми ::=
оголошення_підпрограми
| тіло_підпрограми
| оголошення_типу
| оголошення_підтипу
| оголошення_константи
| оголошення_змінної
| оголошення_псевдоніма

Всі об'єкти даних, типи, підпрограми і т.д., які оголошені у розділі оголошень тіла підпрограми, можуть використовуватися лише локально в межах її тіла. Ці елементи невидимі ззовні підпрограми, але видимі в локально оголошених підпрограмах. Крім того, ці елементи скривають будь-які елементи з тими ж самими назвами, які оголошені поза підпрограмою.

Коли підпрограма викликана, оператори в її тілі виконуються або до кінця тіла підпрограми, або до оператора **return**. Його синтаксис такий:

оператор_повернення ::= **return** [вираз] ;

Якщо оператор **return** знаходиться в тілі процедури, він не повинен мати виразу. У тілі функції обов'язково повинен бути хоч один оператор **return**, виконання функції повинне закінчуватися саме цим оператором, і він повинен мати вираз, значення якого буде повертати функція.

Функції не можуть мати ніяких побічних ефектів. Це означає, що ніяка змінна, яка оголошена поза тілом функції, не може бути змінена функцією. Також функція не може прочитати такі змінні. Таким чином, якщо функція викликається декілька разів з однаковими фактичними параметрами, результат, який вона повертає, буде завжди один і той самий. (У VHDL можуть існувати також функції, які можуть читати та змінювати змінні, які оголошені поза тілом функції. Такі функції оголошуються з використанням слова **impure**. В цьому посібнику вони не будуть розглядатися. Параметри функції можуть мати режим тільки **in**, а не **out** чи **inout**.

Приклад тіла функції:

function byte_to_int(byte : word_8) **return** integer **is**

```
        variable result : integer := 0;
begin
        for index in 0 to 7 loop
                result := result*2 + bit'pos(byte(index));
        end loop;
        return result;
end byte_to_int;
```

3.2.2. Перевантаження

VHDL дозволяє двом підпрограмам мати одну і ту ж саму назву, а відрізнятися лише кількістю та/чи типом формальних параметрів. В таких випадках кажуть, що підпрограма *перевантажена (overloaded)*. Коли викликається така підпрограма, використовується інформація про кількість, тип, порядок фактичних параметрів, а також про імена формальних параметрів (якщо використовується іменована відповідність формальних та фактичних параметрів), для того, щоб визначити, яку підпрограму використати. Якщо викликається функція, використовується також інформація про тип результату.

Наприклад, припустимо, що оголошено дві функції:

```
function check(value : integer) return boolean;
function check(value : word_32) return boolean;
```

Тоді, яка з функцій викликається, залежить від типу (integer чи word_32) фактичного параметра. Так, оператор

```
test := check(1000);
```

викликає першу функцію, а

```
test := check(X"FFF0_0FFF");
```

викликає другу функцію.

Ім’я підпрограми може бути не тільки ідентифікатором, який визначив користувач, а й рядком, який містить один з символів операторів, які перераховані у розділі 1.8. Це дозволяє визначити ці оператори для операндів довільних типів. Наприклад, операцію додавання можна перевантажити так, щоб вона додавала операнди типу word_32:

```
function "+" (a, b : word_32) return word_32 is
begin
        return int_to_word_32( word_32_to_int(a) + word_32_to_int(b) );
end "+";
```

Для того, щоб додати цілі числа, у тілі цієї функції використовується стандартна операція додавання, наперадвизначена в мові. Однак, у виразі:

```
X"1111_0010" + X"1111_FFD0"
```

буде викликатися оголошена програмістом функція, тому що операнди мають тип word_32. Зауважте, що також можна викликати цю функцію з використанням звичайної префіксної системи позначень, як і при виклику звичайної підпрограми, наприклад:

```
"+" (X"1000_0010", X"0000_FFD0")
```

3.2.3. Пакети і тіла пакетів

Пакет є сукупністю оголошень типів, підтипів, констант, підпрограм, псевдонімів, сигналів, компонентів і призначений для інкапсуляції даних та приховування інформації. Зокрема, тіла підпрограм, значення констант і т.і. можуть бути сховані від користувачів пакета, які можуть працювати тільки з їх інтерфейсами, зробленими видимими. Часто в пакетах розташовують оголошення, які потім використовуються в різних об'єктах, архітектурних тілах чи інших пакетах.

Пакет складається з двох частин: оголошення пакета, яке визначає його інтерфейс, та тіла пакета, яке описує його реалізацію. Тіло може бути відсутнім, якщо пакет не має ніяких підпрограм. Синтаксис оголошення пакета такий :

```
оголошення_пакета ::=
        package ідентифікатор is
                розділ_оголошень_пакета
        end [ ідентифікатор] ;
розділ_оголошень_пакета ::= { елемент_оголошення_пакета }
елемент_оголошення_пакета ::=
        оголошення_підпрограми
        | оголошення_типу
        | оголошення_підтипу
        | оголошення_константи
        | оголошення_псевдоніма
        | оголошення_компонента
        | оголошення_сигналу
        | специфікатор_використання
```

Оголошення пакета визначає підпрограми, типи, константи та інші об'єкти, які будуть видимими користувачам пакета, і які також видимі у тілі пакета. (Також в пакеті можуть існувати приватні оголошення, але вони не будуть розглянуті в цьому посібнику.)

Приклад оголошення пакета:

```
package data_types is
        subtype address is bit_vector(24 downto 0);
```

```
        subtype data is bit_vector(15 downto 0);
        constant vector_table_loc : address;
        function data_to_int(value : data) return integer;
        function int_to_data(value : integer) return data;
end data_types;
```

У цьому прикладі значення константи vector_table_loc та тіла двох функцій повинні бути описані у тілі пакета.

Синтаксис тіла пакета такий:

```
тіло_пакета ::=
        package body просте_ім'я_пакета is
                розділ_оголошень_тіла_пакета
        end [ просте_ім'я_пакета ] ;
розділ_оголошень_тіла_пакета ::=
        { елемент_оголошення_тіла_пакета }
елемент_оголошення_тіла_пакета ::=
        оголошення_підпрограми
        | тіло_підпрограми
        | оголошення_типу
        | оголошення_підтипу
        | оголошення_константи
        | оголошення_псевдоніма
        | специфікатор_використання
```

Зауважте, що тіла підпрограми включаються у тіло пакета, а оголошення їх інтерфейсу включаються в оголошення пакета.

Тіло оголошеного вище пакета data_types може мати такий вигляд:

```
package body data_types is
        constant vector_table_loc : address := X"FFFF00";
        function data_to_int(value : data) return integer is
                --оператори функції data_to_int
        end data_to_int;
        function int_to_data(value : integer) return data is
                --оператори функції int_to_data
        end int_to_data;
end data_types;
```

У цьому тілі пакета описані тіла функцій та визначено значення константи. Оголошення підтипів не повторюються, оскільки вони були зроблені в пакеті і є видимими в його тілі.

В наступному прикладі в тілі пакета оголошуються компоненти – логічні елементи:

```
library IEEE;
use IEEE.std_logic_1164.all;
package gates is
        component andg
```

```
        generic (       tpd_hl : time := 1 ns;
                        tpd_lh : time := 1 ns);
        port (          in1, in2 : std_ulogic;
                        out1 : out std_ulogic);
    end component;
    component org
        generic (       tpd_hl : time := 1 ns;
                        tpd_lh : time := 1 ns);
        port (          in1, in2 : std_logic;
                        out1 : out std_logic);
    end component;
    component xorg
        generic (       tpd_hl : time := 1 ns;
                        tpd_lh : time := 1 ns);
        port (          in1, in2 : std_logic;
                        out1 : out std_logic);
    end component;
end gates;
```

3.2.4. Використання пакетів та видимість імен

Після того, як пакет створений, підпрограми, типи та т.і., які оголошені в ньому, можуть використовуватися, якщо перед їх іменем поставити префікс, який складається з імені пакета. Наприклад, елементи, які оголошені в пакеті data_types, який описаний в розділі 3.2.3, можна використовувати в такий спосіб:

```
variable PC : data_types.address;
int_vector_loc := data_types.vector_table_loc + 4*int_level;
offset := data_types.data_to_int(offset_reg);
```

Часто у модулі проекту зручно звертатися до підпрограм та типів, які описані в пакеті, без використання префіксу. Це можна робити, якщо перед модулем поставити специфікатор використання:

```
специфікатор_використання ::= use вибране_ім'я { , вибране_ім'я } ;
вибране_ім'я ::= префікс . суфікс
```

Дія цього специфікатора полягає в тому, що всі перераховані в ньому назви можуть згодом використовуватися без префіксу. Якщо назви всіх об'єктів та типів, які оголошені в пакеті, повинні використовуватися таким чином, можна використати спеціальний суфікс **all**, наприклад:

```
use data_types.all;
use IEEE.std_logic_1164.all;
```

3.3. Процеси та оператор wait

Основна одиниця поведінкового опису в мові VHDL – процес (**process**). Процес – це обмежена група *послідовних* операторів, виконання якої ініціюється вказаними змінами у стані системи. Коли більше, ніж один процес ініційовано в один і той самий час, вони виконуються *одночасно*. Все тіло процесу виконується за один крок моделювання. Процес визначається в операторі процесу з таким синтаксисом:

оператор_процесу ::=
    [мітка_*процесу* :]
        process [(список_чутливості)]
            розділ_оголошень_процесу
        begin
            операторна_частина_процесу
        end process [мітка_*процесу*] ;
розділ_оголошень_процесу ::= { елемент_оголошення_процесу }
елемент_оголошення_процесу ::=
    оголошення_підпрограми
    | тіло_підпрограми
    | оголошення_типу
    | оголошення_підтипу
    | оголошення_константи
    | оголошення_змінної
    | оголошення_псевдоніма
    | специфікатор_використання
операторна_частина_процесу ::= { послідовний_оператор }
послідовний_оператор ::=
    оператор_призупинення_процесу
    | оператор_повідомлення
    | оператор_присвоювання_сигналу
    | оператор_присвоювання_змінній
    | оператор_виклику_підпрограми
    | умовний_оператор
    | оператор_вибору
    | оператор_циклу
    | оператор_наступної_ітерації
    | оператор_виходу
    | оператор_повернення
    | порожній_оператор

Оператор процесу є паралельним і може використовуватися в архітектурі або блоці. Розділ оголошень процесу визначає об’єкти даних, які використовуються локально в межах процесу. Зауважте, що змінні, які оголошені в процесі, зберігають свої значення між активаціями процесу і можуть використовуватися,

щоб зберегти стан системи. Змінні, які оголошені в одному процесі архітектурного тіла, невидимі в інших процесах. (У VHDL існують також спільні змінні, які видимі в декількох процесах. В цьому посібнику вони не розглядаються.)

Процес може містити ряд операторів присвоювання сигналу, ці оператори разом формують драйвер сигналу. У більшості випадків (але не завжди) сигнал має тільки один драйвер і, відповідно, тільки один процес визначає значення сигналу.

Процес активується протягом стадії ініціалізації моделювання. При цьому виконуються всі послідовні оператори, які складають процес, і потім виконання процесу повторюється, починаючи знову з його першого оператора.

Процес може призупинити себе, якщо від виконає оператор **wait**:

оператор_призупинення_процесу ::=
    wait [специфікатор_чутливості] [специфікатор_умови]
    [специфікатор_часу] ;
специфікатор_чутливості ::= **on** список_чутливості
список_чутливості ::= ім'я_*сигналу* { , ім'я_*сигналу* }
специфікатор_умови ::= **until** умова
специфікатор_часу ::= **for** вираз_*часу*

Список чутливості оператора **wait** визначає набір сигналів, до яких процес є чуттєвим, коли він призупинений. Коли будь-який з цих сигналів змінюється, виконання процесу відновляється, і він перевіряє умову. Якщо ця умова істинна (або взагалі відсутня), виконання процесу продовжується з наступного оператора (або з початку, якщо оператор **wait** був останнім), інакше процес призупиняється знову. Якщо список чутливості відсутній, то процес є чуттєвим до всіх сигналів, які згадані в умові. Вираз часу повинен мати позитивне значення, він вказує максимальний час, на який процес може бути призупинений цим оператором. Якщо цей вираз відсутній, то процес може бути призупиненим неозначено довго.

Якщо у заголовок оператора процесу включений список чутливості, то робота процесу така, як у випадку, коли він має наприкінці його операторної частини оператор **wait**. Список чутливості цього уявного оператора такий же самий, як і в заголовку процесу. В такому випадку процес не може мати жодного оператора **wait**.

Приклад процесу зі списком чутливості:

```
process (reset, clock)
        variable state : bit := false;
begin
        if reset then
                state := false;
        elsif clock = true then
                state := not state;
        end if;
        q <= state after prop_delay;
        -- уявний оператор wait, чутиливий до змін сигналів reset, clock
end process;
```

Протягом стадії ініціалізації моделювання процес виконується, і сигналу q присвоюється початкове значення. Після цього процес призупиняється в місці, де в цьому прикладі розташований коментар. Зміна сигналу clock чи reset приводить до відновлення процесу, і його виконання продовжується з початку.

В наступному прикладі процес використовується для створення поведінкового опису дворозрядного лічильника, який рахує передні фронти сигналу ck. Змінна cnt зберігає поточний стан лічильника.

```
process (ck)
        variable cnt : natural := 0;
begin
        if clock = '1' then
                cnt := (cnt + 1) mod 4;
                q0 <= bit'val(cnt mod 2);
                q1 <= bit'val(cnt / 2);
        end if;
end process;
```

Вихідні сигнали лічильника (q0, q1) формуються за допомогою атрибута val, який можна використовувати з будь-яким типом перерахування.

Наступний приклад описує поводження пристрою синхронізації, який використовується в асинхронній логіці. На початку моделювання вихід (сигнал q) дорівнює '0' (значення по замовчуванню). Вихід залишається в цьому стані доти, доки на обох входах на встановлюється значення '1', і тоді приймає значення '1'. Вихід залишається в цьому стані, поки обидва входи не стануть '0', і тоді вихід змінюється назад до '0'.

```
mlr : process
        begin
                wait until a = '1' and b = '1';
```

```
        q <= '1';
        wait until a = '0' and b = '0';
        q <= '0';
end process mlr ;
```

Цей процес не має списку чутливості, тому для його призупинення та відновлення використовуються два оператори **wait**. Обидва оператори **wait** чутливі до змін сигналів a та b, оскільки вони входять в умови відновлення процесу.

3.4. Присвоювання сигналу

Оператор присвоювання сигналу планує одну чи більшу кількість трансакції сигналу (чи порту).

```
оператор_присвоювання_сигналу ::= адресат <= [ transport ]
        форма_сигналу ;
адресат ::= ім'я | агрегат
форма_сигналу ::= елемент_форми_сигналу
        { , елемент_форми_сигналу }
елемент_форми_сигналу ::=
        вираз_значення_сигналу [ after вираз_часу ]
                | null [ after вираз_часу ]
```

Адресат повинний бути чи сигналом, чи агрегатом сигналів (як і при присвоюванні значення змінній, див. розділ 3.1.1). Якщо вираз часу для затримки опущений, то по замовчуванню використовується величина 0. Це означає, що трансакція буде намічена на той самий час, коли виконується оператор присвоювання, але буде виконана протягом наступного циклу моделювання (з дельта-затримкою).

Кожен процес, в якому є оператор присвоювання сигналу, створює *один* драйвер цього сигналу (навіть якщо в процесі стоять декілька операторів присвоювання значення цьому сигналу), який містить в собі список трансакцій, які визначають майбутні значення сигналу. Зауважте, що і у випадку, коли оператор присвоювання фактично не виконується, відповідний драйвер все одно буде створено. В більшості випадків сигнал повинен мати тільки один драйвер, тобто тільки один процес повинен ним керувати. Однак, у мові VHDL сигналу дозволяється мати і декілька драйверів, але в цьому випадку тип такого сигналу повинен дозволяти *розв'язання конфліктів* (див. розділ 5.1)

Оператор присвоювання сигналу додає інформацію про трансакцію, яка планується, до драйвера. Так, наприклад, оператор присвоювання сигналу:

```
s <= '0' after 20 ns;
```

планує присвоєння сигналу s значення '0' через 20 ns після виконання цього оператора. Якщо вищезгаданий оператор виконувався в момент часу 10 ns, то трансакція буде запланована на момент 30 ns. Коли час моделювання досягає 30 ns, ця трансакція буде оброблена, і сигналу буде присвоєне заплановане значення, в цьому прикладі '0'.

Припустимо, в момент часу 32 ns виконується оператор:

```
s <= '1' after 8 ns, '0' after 40 ns;
```

До драйвера будуть додані дві нові трансакції: в момент 40 ns сигнал повинен прийняти значення '1', а в момент 72 ns – значення '0'. Слід відзначити, що, коли декілька трансакцій перераховані в операторі присвоювання сигналу, затримки, які їм відповідають, повинні зростати.

Якщо виконується оператор присвоювання сигналу, а у драйвері існують раніше заплановані трансакції, то деякі з них можуть бути вилучені. Як це робиться, залежить від того, чи використане слово **transport** в операторі нового присвоювання. Якщо це слово є, кажуть, що використовується *транспортна затримка (transport delay)*. У цьому випадку всі трансакції, які були заплановані до виконання оператора з транспортною затримкою на час після його першої трансакції, вилучаються, і після цього додаються трансакції, які відповідають оператору, який виконується. Тобто, нові трансакції подавляють старі. Так, продовжуючи приклад, розглянемо випадок, коли у момент 36 ns виконується оператор

```
s <= transport 'Z' after 20 ns;
```

Тоді трансакція, запланована на час 72 ns буде вилучена, і драйвер буде містити інформацію про дві заплановані трансакції: при 40ns – '1', при 56 ns – 'Z'.

Інший вид затримки, *інерційна затримка (inertial delay)*, використовується, щоб моделювати пристрої, які не реагують на вхідні імпульси коротші, ніж їхні затримки. Якщо слово **transport** в операторі присвоювання сигналу відсутнє, буде використана саме інерційна затримка. Коли інформація про трансакцію з інерційною затримкою додається до драйвера сигналу, то, по-

перше, як і у випадку транспортної затримки, всі старі трансакції, заплановані на час після нової трансакції, вилучаються, і додається нова трансакція. По-друге, досліджуються всі старі трансакції, які були заплановані раніше на час перед новою трансакцією. Якщо є старі трансакції зі значеннями, які відрізняються від значення нової трансакції, то всі вони також вилучаються. Залишаються тільки старі трансакції з тим же самим значенням, як і нова.

Щоб пояснити це, припустимо, що у момент часу 0 ns був створений драйвер сигналу s з таким змістом: 30ns – '1', 45ns – '0', 60ns – '1', 90ns – 'Z', а в момент 10 ns виконується оператор

```
s <= '1' after 60 ns;
```

Після виконання цього оператора у драйвері сигналу s буде міститися інформація про дві трансакції: 60 ns – '1', 70 ns – '1'.

Коли декілька трансакцій перераховано в операторі присвоювання сигналу з інерційною затримкою, то тільки перша трансакція має інерційну затримку; інші трансакції мають транспортну затримку.

3.5. Паралельний оператор присвоювання сигналу

Часто процес, який створює драйвер сигналу, містить тільки оператор присвоювання сигналу. В VHDL існує зручний засіб для опису таких процесів. Це оператор паралельного присвоювання сигналу:

```
паралельний_оператор_присвоювання_сигналу ::=
        [ мітка : ] оператор_умовного_присвоювання_сигналу
        | [ мітка : ] оператор_вибіркового_присвоювання_сигналу
```

Для кожного виду паралельного присвоювання сигналу, існує відповідний процес, який робить теж саме.

3.5.1. Умовне присвоювання сигналу

Оператор умовного присвоювання сигналу – це короткий запис процесу, який містить в собі лише один умовний оператор, в гілках якого стоять тільки оператори присвоювання одному сигналу. Його синтаксис такий:

```
оператор_умовного_присвоювання_сигналу ::=
        адресат <= опції умовна_форма_сигналу ;
опції ::= [ guarded ] [ transport ]
умовна_форма_сигналу ::=
        { форма_сигналу when умова else }
```

```
        форма_сигналу
```

Використання слова **guarded** буде розглянуто у розділі 5.6. Якщо цей оператор включає слово **transport**, то присвоювання сигналу в еквівалентному процесі буде виконуватися з транспортною затримкою.

Розглянемо таке умовне присвоювання сигналу:

```
s <=    форма_сигналу _1 when умова_1 else
        форма_сигналу _2 when умова_2 else
        …
        форма_умова_n;
```

Еквівалентний процес має вигляд:

```
process
        if умова_1 then
                s <= форма_сигналу _1;
        elsif умова_2 then
                s <= форма_сигналу _2;
        elsif …
        else
                s <= форма_сигналу _n;
wait [ список_чутливості ];
end process;
```

Якщо жодна з форм сигналів чи умов не містить посилання на сигнал, то оператор **wait** наприкінці еквівалентного процесу не має списку чутливості. Це означає, що після того, як присвоювання зроблене, процес зупиняється назавжди. Наприклад, якщо short_pulse_required – константа, то оператор умовного присвоювання

```
reset <= '1', '0' after 10 ns when short_pulse_required else
        '1', '0' after 50 ns;
```

планує дві трансакції сигналу reset і зупиняється на весь наступний час моделювання.

З іншого боку, якщо у формах сигналів чи умовах існують посилання на сигнали, то оператор **wait** наприкінці еквівалентного процесу має список чутливості, який складається з усіх сигналів, на які є посилання. Так, оператор умовного присвоювання

```
mux_out <= 'Z' after Tpd when en = '0' else
        in_0 after Tpd when sel = '0' else
        in_1 after Tpd;
```

є чуттєвим до сигналів in_0, in_1, en та sel. Цей оператор виконується протягом стадії ініціалізації моделювання та всякий раз після зміни хоч одного з вказаних сигналів.

Умова в операторі умовного присвоювання сигналу може бути відсутньою. Такий оператор є еквівалентом процесу, який містить тільки один оператор присвоювання сигналу. Таким чином, оператор

```
s <= форма_сигналу ;
```

є еквівалентним процесу

```
process
        s <= форма_сигналу ;
        wait [ список_чутливості ];
end process;
```

в список чутливості якого входять всі сигнали, які є в правій частині оператора присвоювання значення сигналу.

3.5.2. Вибіркове присвоювання сигналу

Оператор вибіркового присвоювання сигналу – це короткий запис процесу, який містить тільки один оператор вибору, в гілках якого стоять тільки оператори присвоювання одному сигналу. Його синтаксис такий:

```
оператор_вибіркового_присвоювання_сигналу ::=
        with вираз select
                адресат <= опції selected_waveforms ;
вибіркова_форма_сигналу ::=
        { форма_сигналу when список_виборів, }
        форма_сигналу when список_виборів
список_виборів ::= вибір { | вибір }
```

Специфікатори присвоювання (опції) в цьому операторі такі самі, як і в операторі умовного присвоювання сигналу. Так, якщо оператор включає слово **transport**, то присвоювання сигналу в еквівалентному процесі буде виконуватися з використанням транспортної затримки.

Для оператора вибіркового присвоювання сигналу:

```
with вираз select
        s <=    форма_сигналу_1 when список_виборів_1,
                форма_сигналу_2 when список_виборів_2,
                …
                форма_сигналу_n when список_виборів_n;
```

еквівалентний процес буде мати вигляд:

```
process
        case вираз is
                when список_виборів_1=>
                        s <= форма_сигналу_1;
                when список_виборів_2=>
                        s <= форма_сигналу_2;
```

```
        …
        when список_виборів_n=>
                s <= форма_сигналу_n;
    end case;
    wait [ список_чутливості ];
end process;
```

Список чутливості оператора **wait** в еквівалентному процесі визначається таким же чином, як і у випадку оператора умовного присвоювання сигналу. Тобто, якщо ніякі сигнали не згадані в виразі вибору чи у формах сигналів, то оператор **wait** не має списку чутливості. В інших випадках список чутливості містить усі сигнали, які згадані в виразі вибору чи в формах сигналів. Елементи списків виборів у цьому операторі повинні задовольняти там же умовам, як і в послідовному операторі case (див. розділ 3.1.3), тобто повинні бути локально статичними.

Приклад оператора вибіркового присвоювання сигналу:

```
with alu_function select
    alu_result <=   op1 + op2 when alu_add | alu_incr,
                    op1 – op2 when alu_subtract,
                    op1 and op2 when alu_and,
                    op1 or op2 when alu_or,
                    op1 and not op2 when alu_mask;
```

В цьому прикладі значення сигналу alu_function використовується, щоб вибрати один з виразів, який буде присвоєний сигналу alu_result. Оператор чутливий до сигналів alu_function, op1 і op2, тому, кожний раз, коли один з цих сигналів змінюється, виконується присвоювання сигналу alu_function.

4. Організація моделі

В попередніх главах були описали різні засоби мови VHDL. Мета цієї глави – показати, як всі вони використовуються разом, щоб створити закінчений опис цифрової системи.

4.1. Модулі проекту та бібліотеки

При створенні системи користувач може використовувати п’ять різних описів – оголошення об’єктів, їх архітектурні тіла, пакети та їх тіла, оголошення конфігурацій (див. розділ 4.2). Вони можуть бути проаналізовані незалежно і називаються модулями проекту. Коли створюється опис системи на мові VHDL, текст модулів записують у файл проекту. Потім викликається компілятор, який виконує синтаксичний та семантичний *аналіз (analysis)*, створює описи модулів у проміжній формі і зберігає їх у бібліотеці проекту. Часто модуль може бути проаналізований для наступного використання в різних проектах. Такий модуль називають бібліотечним. *Первинними (primary)* модулями проекту являються оголошення об’єктів, пакетів і конфігурацій. *Вторинні (secondary)* модулі – це архітектурні тіла та тіла пакетів. Вторинні модулі залежать від специфікації їх інтерфейсів, які наведені в первинних модулях, так що первинні модулі повинні бути проаналізовані перед будь-якими відповідними вторинними.

Якщо всі модулі проекту проаналізовані, то з використанням відповідних програмних засобів можна виконати моделювання системи. Перед цим відбувається *компонування* всіх модулів проекту. Саме на цьому етапі зв’язуються декларації об’єктів та їх архітектурні тіла, замість екземплярів компонентів підставляються об’єкти, які вказані в описі конфігурації, і створюється повний ієрархічних опис системи, який можна промоделювати.

Структура файлу проекту повинна відповідати такому синтаксичному правилу:

```
файл_проекту ::= модуль_проекту { модуль_проекту }
модуль_проекту ::= специфікатор_контексту бібліотечний_модуль
специфікатор_контексту ::= { одиниця_контексту }
одиниця_контексту ::= специфікатор_бібліотеки
        | специфікатор_використання
специфікатор_бібліотеки ::= library список_логічних_імен ;
```

список_логічних_імен ::= логічне_ім'я { , логічне_ім'я }
бібліотечний_модуль ::= первинний_модуль | вторинний_модуль
первинний_модуль ::=
оголошення_об'єкта | оголошення_конфігурації
| оголошення_пакета
вторинний_модуль::= архітектурне_тіло | тіло_пакета

Бібліотеки складаються з раніше проаналізованих модулів. До бібліотек звертаються за допомогою логічних імен. Це ім'я засобами операційної системи повинно бути співставлено імені, під яким бібліотека зберігається операційною системою. Наприклад, кожна бібліотека може зберігатися як окремий файл, і логічна назва використовується, щоб визначити ім'я цього файлу, хоч ім'я файлу не обов'язково співпадає з логічним іменем. Імена директорій та файлів, в яких зберігається бібліотека, їх структура на визначаються стандартом VHDL, а залежать від реалізації. До бібліотечних модулів можна звертатися з використанням повного імені модуля, яке складається з префіксу, який є логічним іменем бібліотеки, та імені модуля. Так, наприклад, ttl_lib.ttl_10 є посиланням до модуля з іменем ttl_10 у бібліотеці ttl_lib.

Специфікатор контексту, який передує опису кожного модуля, вказує на бібліотеки та пакети, які використовує цей модуль. Область видимості назв, які робляться видимими описом контексту, простирається тільки до кінця модуля, який розташований безпосередньо після опису контексту. Специфікатор бібліотеки вказує бібліотеки, які можна використовувати в модулі, а специфікатор використання дозволяє звертатися до бібліотечних модулів без префіксу.

Існує дві спеціальні бібліотеки, які доступні в усіх модулях проекту. Ці бібліотеки не потрібно вказувати в специфікаторі **library**. Перша з них називається work, вона є робочою бібліотекою проекту. В цій бібліотеці компілятор VHDL розташовує всі проаналізовані модулі проекту. Отже, всі раніше проаналізовані модулі проекту можуть бути використані в нових модулях цього ж проекту за допомогою префікса work.

Інша спеціальна бібліотека називається std, і вона складається з двох пакетів: standard та textio. Пакет standard містить в собі опис всіх напередвизначених типів. Всі елементи цього пакета видимі по замовчуванню, так що для звертання до них не потрібно використовувати специфікатор використання. В

пакеті textio розташовані засоби для роботи з текстовими файлами, які не розглядаються в цьому посібнику.

В багатьох проектах використовується бібліотека IEEE, яка не входить в стандарт VHDL. Вона повинна бути вказана в описі контексту. В цій бібліотеці знаходиться пакет std_logic_1164, в якому описані типи багатозначної логіки, які узагальнюють тип bit, та функції для роботи з ними. Це типи std_logic і std_logic_vector (див. розділ 1.7.4), які дозволяють розв'язання конфліктів (див. розділ 5.1), для них описана відповідна функція розв'язання конфліктів, та типи std_ulogic, std_ulogic_vector, які не дозволяють існування конфліктів. Також у цьому пакеті описані їх підтипи. Для використання цих типів без префіксу описують відповідний контекст:

```
library IEEE;
use IEEE.std_logic_1164.all;
```

4.2. Конфігурації

У розділах 2.2.3 і 2.2.4 було показано, як при створенні структурного опису системи спочатку оголошується компонент, а потім створюються їх екземпляри. Оголошення компонента вказує інтерфейс, якому повинен задовольняти об'єкт, який буде використовуватися в якості екземпляра компонента. Якщо об'єкт описаний в тому ж файлі, в якому він буде використовуватися в якості екземпляра компонента, і він має ім'я, кількість та тип портів, які співпадають з іменем компонента, кількістю та типом його портів, то описувати конфігурацію не обов'язково. В протилежному випадку реальний об'єкт, який буде використовуватися в якості екземпляра компонента, вказується в оголошенні конфігурації. Оголошення конфігурації може також використовуватися, щоб визначити параметри настроювання для компонентів і блоків. Об'єкт може мати декілька архітектурних тіл, інформація про те, яке з них потрібно використовувати, наводиться також у конфігурації. Кожне оголошення конфігурації відповідає одному об'єкту. Таким чином, оголошення конфігурації відіграє важливу роль при створенні опису конструкції.

```
оголошення_конфігурації ::=
        configuration ідентифікатор of ім'я_об'єкта is
                розділ_оголошень_конфігурації
                конфігурація_блоку
```

end [просте_ім'я_*конфігурації*] ;
розділ_оголошень_конфігурації ::=
{ елемент_оголошення_конфігурації }
елемент_оголошення_конфігурації ::= специфікатор_використання
конфігурація_блоку ::=
for специфікація_блоку
{ специфікатор_використання}
{ одиниця_конфігурації }
end for ;
специфікація_блоку ::= ім'я_*архітектурного_тіла* | мітка_*блоку*
одиниця_конфігурації ::= конфігурація_блоку | конфігурація_компонента
конфігурація_компонента ::=
for специфікація_компонента
[**use** опис_зв'язування ;]
[конфігурація_блоку]
end for ;
специфікація_компонента ::= список_екземплярів : ім'я_*компонента*
список_екземплярів ::=
мітка_*екземпляра* { , мітка_*екземпляра*)
| **others**
| all
опис_зв'язування ::=
розділ_співставлення_об'єктів
[розділ_співставлення_параметрів_настроювання]
[розділ_співставлення портів]
розділ_співставлення_об'єктів ::=
entity ім'я_*об'єкта* [(ідентифікатор_*архітектурного_тіла*)]
| **configuration** ім'я_*конфігурації*
| **open**
розділ_співставлення_параметрів_настроювання ::=
generic map
(список_співставлення_*параметрів_настроювання*)
розділ_співставлення портів ::= **port map**
(список_співставлення_*портів*)

Розділ оголошень конфігурації дозволяє використовувати в ній елементи з пакетів. Зовнішній блок в оголошенні конфігурації визначає конфігурацію для архітектурного тіла вказаного об'єкта. Наприклад, у главі 2 був розглянутий приклад об'єкта processor та його архітектурного тіла, який тут повторений у дещо спрощеному вигляді:

```
entity processor is
        generic (max_clock_speed : frequency := 30 MHz);
        port ( port list );
end processor;

architecture block_structure of processor is
```

```
        розділ оголошень
begin
        control_unit : block
                port ( port list );
                port map ( association list );
                розділ оголошень блоку control_unit
        begin
                оператори блоку control_unit
        end block control_unit;
        data_path : block
                port ( port list );
                port map ( association list );
                розділ оголошень блоку data_path
        begin
                оператори блоку data_path
        end block data_path;
end block_structure;
```

Оголошення конфігурації test_config для цього архітектурного тіла може мати такий вигляд:

```
configuration test_config of processor is
        use work.processor_types.all
        for block_structure
                конфігурація архітектури block_structure
        end for;
end test_config;
```

У цьому прикладі зміст пакета processor_types, який знаходиться у робочій бібліотеці, зроблений видимим, і описується конфігурація, яка відноситься до архітектури block_structure об'єкта processor.

У межах конфігурації архітектурного тіла можуть бути описані конфігурації його субмодулів, тобто блоків та компонентів. В нашому прикладі конфігурації обох блоків можуть бути описані за допомогою вкладених конструкцій **for - end for**:

```
configuration test_config of processor is
        use work.processor_types.all
        for block_structure
                for control_unit
                        конфігурація блоку control_unit
                end for;
                for data_path
                        конфігурація блоку data_path
                end for;
        end for;
end test_config;
```

Для компонентів конфігурація використовується, щоб вказати, який об'єкт слід використовувати для конкретного екземпляра компонента. Припустимо, що блок data_path у вищезгаданому прикладі складається з екземплярів компонента alu:

```
data_path : block
        port ( port list );
        port map ( association list );
        component alu
                port (function : in alu_function;
                        op1, op2 : in bit_vector_32;
                        result : out bit_vector_32);
        end component;
        інші оголошення блоку data_path
begin
        data_alu : alu
                port map (function => alu_fn, op1 => b1, op2 => b2,
                        result => alu_r);
        інші оператори блоку data_path
end block data_path;
```

Припустимо також, що у бібліотеці project_cells є об'єкт, який називається alu_cell, оголошений таким чином:

```
entity alu_cell is
        generic (width : positive);
        port (  function_code : in alu_function;
                operand1, operand2 : in bit_vector(width-1 downto 0);
                result : out bit_vector(width-1 downto 0);
                flags : out alu_flags);
end alu_cell;
```

який має архітектуру behaviour. Цей об'єкт відповідає оголошенню компонента alu, тому що вибором відповідного значення параметра настроювання width можна зробити так, що тип його портів operand1, operand2 та result буде відповідати типу портів оголошення компонента alu, а порт flags можна залишити непід'єднаним. Таким чином, цей об'єкт можна використати в якості екземпляра компонента alu. Про необхідність використовувати об'єкт alu_cell в якості екземпляра data_alu компонента alu можна вказати за допомогою такої конфігурації блоку data_path:

```
for data_path
        for data_alu : alu
                use entity project_cells.alu_cell(behaviour)
                        generic map (width => 32)
                        port map (function_code => function,
```

```
                    operand1 => op1,operand2 => op2,
                    result => result, flags => open);
    end for;
    продовдження конфігурації блоку data_path
end for;
```

Наприкінці розглянемо випадок, коли у бібліотеці описана конфігурація alu_struct архітектурного тіла structure об'єкта alu_cell. У конфігурації блоку можна вказати, що потрібно використовувати саме цю конфігурацію для екземпляра data_alu компонента alu:

```
for data_path
    for data_alu : alu
        use configuration project_cells.alu_struct
            generic map (width => 32)
            port map (function_code => function,
                operand1 => op1,operand2 => op2,
                result => result, flags => open);
    end for;
    продовження конфігурації блоку data_path
end for;
```

4.3. Приклад проекту

Розглянемо закінчений файл проекту, в якому розробляється багаторозрядний суматор, кількість розрядів якого визначається параметром настроювання. Файл проекту містить декілька модулів проекту. Основний тип сигналів, який використовується у цьому проекті – std_logic (див. розділ 1.7.4), тому перед модулями описано відповідний контекст. Перший модуль проекту – оголошення об'єкта adder. Це однорозрядний суматор, який потім буде використано, як компонент багаторозрядного суматора. За ним йдуть два вторинних модуля – архітектурні тіла behaviour та об'єкта adder. Вони повинні задовольняти оголошенню об'єкта adder. В структурному описі використовуються описи логічних елементів, які були раніше проаналізовані і зберігаються в робочій бібліотеці проекту, а їх оголошення розташовані в пакеті gates (див. розділ 3.2.3). Перед їх використанням описано відповідний контекст: **use** work.gates.**all**. Зверніть увагу, що в такому випадку ці компоненти явно не оголошуються в архітектурному тілі. Після цього описано об'єкт багаторозрядного суматора adderN та два його архітектурних тіла. В структурному архітектурному тілі

використовується об’єкт однорозрядного суматора adder, який був описаний вище. Для створення регулярної структури застосовується оператор паралельної генерації, який був розглянутий в розділі 2.2.5. Параметр настоювання N визначає кількість розрядів суматора. Далі йде оголошення іншого первинного модуля, testbench, який є іспитовим стендом для суматора. Після нього приведено відповідний вторинний модуль – архітектурне тіло іспитового стенда. В ньому створено екземпляр багаторозрядного суматора, а для генерації тестових сигналів та аналізу результатів використовується процес. Результати моделювання повідомляються користувачу за допомогою операторів повідомлення. Останні модулі – оголошення конфігурацій test_adder_behavioral та test_adder_structural іспитового стенда. В цих конфігураціях використовуються раніше визначені модулі проекту, які знаходяться у робочій бібліотеці, тому оператор library непотрібен. Зверніть увагу на те, що у цих конфігураціях на об’єкти adderN та adder посилаються як work.adderN та work.adder, використовуючи назву робочої бібліотеки. В круглих дужках вказані архітектурні тіла, які потрібно використовувати для цих об’єктів.

В цьому проекті всі модулі проекту містяться в одному файлі. Однак можна розділити проект на декілька файлів, наприклад, в одному файлі може міститися опис тільки одного модуля проекту. Якщо використовується декілька файлів, то потрібно слідкувати за тим, щоб файли компілювалися в правильному порядку, і перекомпілювати залежні файли, якщо зроблені зміни в одному модулі проекту. Часто програмні засоби моделювання роблять це автоматично.

```
library IEEE;
use IEEE.std_logic_1164.all;
entity adder is
        port (a: in std_logic;
                b: in std_logic;
                cin: in std_logic;
                sum: out std_logic;
                cout: out std_logic);
end adder;


-- булеві функції, які описують роботу однорозрядного суматора
```

```
architecture behaviour of adder is
begin
        sum <= (a xor b) xor cin;
        cout <= (a and b) or (cin and a) or (cin and b);
end behaviour;


-- структурний опис однорозрядного суматора
use work.gates.all;
architecture  of adder is
        signal xor1_out, and1_out, and2_out, or1_out : std_logic;
begin
        xor1: xorg port map(in1  => a, in2  => b, out1 => xor1_out);
        xor2: xorg port map(in1 => xor1_out, in2 => cin, out1 => sum);
        and1: andg port map(in1 => a, in2 => b, out1 => and1_out);
        or1: org port map(in1 => a, in2 => b, out1 => or1_out);
        and2: andg port map(in1 => cin, in2 => or1_out,
                                    out1 => and2_out);
        or2: org port map(in1 => and1_out, in2 => and2_out, out1 => cout);
end ;


--------------------------------------------------------------------------
-- N-розрядний суматор
-- розрядність визначається параметром настроювання N
--------------------------------------------------------------------------
library IEEE;
use IEEE.std_logic_1164.all;
entity adderN is
        generic(N : integer := 16);
        port (    a: in std_logic_vector(N downto 1);
                  b: in std_logic_vector(N downto 1);
                  cin : in std_logic;
                  sum : out std_logic_vector(N downto 1);
                  cout : out std_logic);
end adderN;

-- структурний опис N-розрядного суматора
architecture  of adderN is
        component adder
                port (    a: in std_logic; b: in std_logic; cin : in std_logic;
                          sum : out std_logic; cout : out std_logic);
        end component;
        signal carry : std_logic_vector(0 to N);
begin
        carry(0) <= cin;
        cout <= carry(N);
```

```
        -- створення та з'єднання N екземплярів
        -- однорозрядного суматора
        -- Використовується оператор паралельної
        -- генерації (див розділ 2.2.5)
        gen: for I in 1 to N generate
                add: adder port map(a => a(I), b => b(I), cin => carry(I - 1),
                                        sum => sum(I), cout => carry(I));
                end generate;
end ;


-- поведінковий опис N-розрядного суматора
architecture behaviour of adderN is
begin
        p1: process(a, b, cin)
                variable vsum : std_logic_vector(N downto 1);
                variable carry : std_logic;
        begin
                carry := cin;
                for i in 1 to N loop
                        vsum(i) := (a(i) xor b(i)) xor carry;
                        carry := (a(i) and b(i)) or (carry and (a(i) or b(i)));
                end loop;
                sum <= vsum;
                cout <= carry;
        end process p1;
end behaviour;


entity testbench is
end;

--------------------------------------------------------------------------
-- тестовий стенд для перевірки роботи N-розрядного суматора
--------------------------------------------------------------------------
library IEEE;
use IEEE.std_logic_1164.all;
architecture adder8 of testbench is
        component adderN
                generic(N : integer);
                port (  a: in std_logic_vector(N downto 1);
                        b: in std_logic_vector(N downto 1);
                        cin : in std_logic;
                        sum  :out std_logic_vector(N downto 1);
                        cout : out std_logic);
        end component;
```

```
	constant N : integer := 8;

	signal a : std_logic_vector(N downto 1);
	signal b : std_logic_vector(N downto 1);
	signal cin : std_logic;
	signal sum : std_logic_vector(N downto 1);
	signal cout : std_logic;

	type test_record_t is record
		a :  std_logic_vector(N downto 1);
		b :  std_logic_vector(N downto 1);
		cin :  std_logic;
		sum :  std_logic_vector(N downto 1);
		cout :  std_logic;
	end record;

	type test_array_t is array(positive range <>) of test_record_t;

	-- набір сигналів для перевірки роботи N-розрядного суматора
	constant test_patterns : test_array_t := (
		(a => "00000000", b => "00000001", cin => '0',
		sum => "00000001", cout => '0'),
		(a => "00000001", b => "00000001", cin => '0',
		sum => "00000010", cout => '0'),
		(a => "00000001", b => "00000001", cin => '1',
		sum => "00000011", cout => '0'),
		(a => "00001010", b => "00000011", cin => '0',
		sum => "00001101", cout => '0'),
		(a => "00000011", b => "00001010", cin => '0',
		sum => "00001101", cout => '0'),
		(a => "00000101", b => "00000001", cin => '1',
		sum => "00001000", cout => '0'),
		(a => "00000011", b => "11111100", cin => '0',
		sum => "11111111", cout => '0'),
		(a => "00000011", b => "11111100", cin => '1',
		sum => "00000000", cout => '1'),
		(a => "01010101", b => "01010101", cin => '0',
		sum => "10101010", cout => '0'),
		(a => "00000000", b => "00000000", cin => '0',
		sum => "00000000", cout => '0')
	);

--
-- перетворення типу std_logic тип character
--
	type stdlogic_to_char_t is array(std_logic) of character;
	constant to_char : stdlogic_to_char_t := (
```

```
                'U' => 'U',
                'X' => 'X',
                '0' => '0',
                '1' => '1',
                'Z' => 'Z',
                'W' => 'W',
                'L' => 'L',
                'H' => 'H',
                '-' => '-');

--
-- перетворення типу std_logic_vector в рядок символів
--
        function to_string(inp : std_logic_vector) return string is
                alias vec : std_logic_vector(1 to inp'length) is inp;
                variable result : string(vec'range);
        begin
                for i in vec'range loop
                        result(i) := to_char(vec(i));
                end loop;
                return result;
        end;

begin
        -- створення екземпляра N-розрядного суматора
        uut: adderN generic map(N)
                port map(a => a, b => b, cin => cin, sum => sum,
                                cout => cout);

        -- генерація тестових сигналів та перевірка результатів
        test: process
                variable vector : test_record_t;
                variable found_error : boolean := false;
        begin
                for i in test_patterns'range loop
                        vector := test_patterns(i);
                         -- apply the stimuls
                        a <= vector.a;
                        b <= vector.b;
                        cin <= vector.cin;
                         -- wait for the outputs to settle
                        wait for 100 ns;
                        -- check the results
                        if (sum  /= vector.sum) then
                                assert false report
                                        "Sum is " & to_string(sum)
                                        & ". Expected "
```

```
                                        & to_string(vector.sum);
                                        found_error := true;
                        end if;
                        if (cout /= vector.cout) then
                                assert false report
                                        "Cout is " & to_char(cout) & ". "
                                        & "Expected value is "
                                        & to_char(vector.cout);
                                        found_error := true;
                        end if;
                end loop;
                assert not found_error report
                        "There were ERRORS in the test." severity note;
                assert found_error report "Test completed with no errors."
                        severity note;
                -- наступним оператором процес
                -- зупиняється назавжди
                wait;
        end process;
end;

configuration test_adder_behavioral of testbench is
        for adder8
                for all: adderN
                        use entity work.adderN(behaviour);
                end for;
        end for;
end test_adder_behavioral;

configuration test_adder_structural of testbench is
        for adder8
                for all: adderN
                        use entity work.adderN(structure);
                        for structural
                                -- конфігурування компонентів adder, які
                                -- використані в adderN
                                for gen
                                        for all : adder
                                        use entity work.adder(structure);
                                        end for;
                                end for;
                         end for;
                end for;
        end for;
end test_adder_structural;
```

5. Додаткові засоби мови VHDL

Ця глава описує додаткові засоби, які існують в мові VHDL. Хоча можна створити багато корисних моделей, які використовують тільки частини мови, охоплені у попередніх главах, засоби, які описані в цій главі, істотно спрощують процес створення моделі та збільшують її точність.

5.1. Розв'язання конфліктів

У багатьох цифрових системах використовуються шини, тобто декілька драйверів керують одним сигналом. Наприклад, якщо використовується схема з відкритим колектором чи стоком, коли колекторні або стокові виходи декількох елементів з'єднані між собою і працюють на загальне навантаження, низький рівень сигналу може бути встановлений будь-яким драйвером, а високий рівень сигналу буде тільки, коли всі транзистори закриті. Таку схему називають "монтажним або". З іншого боку, якщо використовуються драйвери з трьома станами, то тільки один драйвер може бути активним в один момент часу, і він визначає значення сигналу.

VHDL звичайно дозволяє існування тільки одного драйвера для кожного сигналу. Щоб моделювати сигнали з декількома драйверами, VHDL використовує поняття типу сигналу, який дозволяє розв'язання конфліктів. Оголошення такого типу включає ім'я *функції розв'язання конфліктів (resolution function)*, яка приймає значення всіх драйверів, що керують сигналом, і на основі цих значень обчислює результуюче значення сигналу.

Тип сигналу, який дозволяє розв'язання конфліктів, оголошується з використанням синтаксису для оголошення підтипу:

опис_підтипу ::= [ім'я_*функції_розв'язання_конфліктів*]
    позначення_типу [обмеження]

Функція розв'язання конфліктів повинна бути попередньо оголошена. Аргументом цієї функції повинен бути необмежений масив з елементами, тип яких співпадає з типом сигналу, а повертати вона повинна результат цього типу.

Розглянемо приклад:

```
type logic_level is (L, Z, H);
```

```
type logic_array is array (integer range <>) of logic_level;
function resolve_logic (drivers : in logic_array) return logic_level;
subtype resolved_level is resolve_logic logic_level;
```

У цьому прикладі тип logic_level представляє три можливих стани для цифрового сигналу: низький рівень (L), високий рівень (H) та високий імпеданс (Z). Підтип resolved_level може використовуватися, щоб оголосити сигнал, для якого можливо розв'язання конфліктів, і який може приймати значення типу logic_level.

Функція розв'язання конфліктів може бути реалізована таким чином:

```
function resolve_logic (drivers : in logic_array) return logic_level;
begin
        for index in drivers'range loop
                if drivers(index) = L then
                        return L;
                end if;
        end loop;
        return H;
end resolve_logic;
```

Ця функція перевіряє всі елементи масиву драйверів, і якщо хоч один з них має значення L, функція повертає L. Якщо всі драйвери мають значення Z чи H, то функція повертає H. Таким чином, за допомогою цієї функції моделюється схема "монтажне або". Зауважте, що в деяких випадках функція розв'язання може бути викликана з аргументом, який є порожнім масивом, і вона повинна коректно обробити такий випадок. Описана вище функція повертає в цьому випадку значення H.

Для типу std_logic (див. розділи 1.7.4 та 4.1), який описано в пакеті std_logic_1164, функція розв'язання конфліктів для двох драйверів визначається такою таблицею істинності:

	'U'	'X'	'0'	'1'	'Z'	'W'	'L'	'H'	'-'
'U'	**'U'**	**'U'**	**'U'**	**'U'**	**'U'**	**'U'**	**'U'**	**'U'**	**'U'**
'X'	**'U'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**
'0'	**'U'**	**'X'**	**'0'**	**'X'**	**'0'**	**'0'**	**'0'**	**'0'**	**'X'**
'1'	**'U'**	**'X'**	**'X'**	**'1'**	**'1'**	**'1'**	**'1'**	**'1'**	**'X'**
'Z'	**'U'**	**'X'**	**'0'**	**'1'**	**'Z'**	**'W'**	**'L'**	**'H'**	**'X'**
'W'	**'U'**	**'X'**	**'0'**	**'1'**	**'W'**	**'W'**	**'W'**	**'W'**	**'X'**
'L'	**'U'**	**'X'**	**'0'**	**'1'**	**'L'**	**'W'**	**'L'**	**'W'**	**'X'**
'H'	**'U'**	**'X'**	**'0'**	**'1'**	**'H'**	**'W'**	**'W'**	**'H'**	**'X'**
'-'	**'U'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**	**'X'**

В першому стовпчику та рядку стоять значення двох драйверів, а значення, яке повертає функція розв'язання конфліктів, знаходиться на перетині відповідного рядка та стовпчика. Коли драйверів декілька, ця функція викликається спочатку для перших двох драйверів, а потім для наступного драйвера та результату попереднього виклику функції.

5.2. Створення екземпляра об'єкта та конфігурації в архітектурному тілі

В стандарті VHDL-93 була введена можливість безпосереднього створення екземпляра об'єкта (та його архітектурного тіла) в архітектурному тілі. Синтаксис оператора створення екземпляра об'єкта подібний до синтаксису оператора створення екземпляра компонента, але перед назвою об'єкта, екземпляр якого створюють, потрібно поставити слово **entity**, а після імені об'єкта в дужках можна вказати ім'я архітектурного тіла. Попереднє оголошення об'єкта, який буде використовуватися в архітектурному тілі, не потрібне. Наприклад:

```
entity test1 is
end test1;

library LIB1;
architecture arch1 of test1 is
begin
        L: entity LIB1.COMP1(Arch_of_lib1) port map ... ;
end arch1 ;
```

Перед використанням об'єкт та його архітектурне тіло повинні бути проаналізовані та розміщені у під'єднаній бібліотеці.

Подібним чином можна поставити ціле дерево модулів проекту, яке описано в конфігурації. Всі модулі проекту, які входять в конфігурацію, повинні буди попередньо проаналізовані.

```
entity test1 is
end test1;

architecture arch1 of test1 is
begin
        L: configuration Work.Config1 ... ;
end arch1;
```

5.3. Паралельні оператори повідомлення та виклику процедури

В мові VHDL є два паралельні оператори, які не були розглянуті у попередніх главах, – це паралельні оператори повідомлення і виклику процедури. Паралельний оператор повідомлення еквівалентний процесу, що містить тільки оператор повідомлення, після якого стоїть оператор wait. Його синтаксис такий:

паралельний_оператор_повідомлення ::=
    [мітка :] оператор_повідомлення

Паралельний оператор повідомлення

```
L : assert умова report рядок_повідомлення severity
    рівень_серйозності;
```

є еквівалентним процесу:

```
L : process
begin
    assert умова report рядок_повідомлення severity
        рівень_серйозності;
    wait [ список_чутливості ] ;
end process L;
```

Список чутливості включає всі сигнали, які входять в умову, яка перевіряється. Якщо ніякі сигнали не входять в цю умову, процес виконується один раз при ініціалізації моделювання, перевіряє умову, і потім зупиняється на весь час моделювання.

Інший паралельний оператор, оператор паралельного виклику процедури, є еквівалентним процесу, який містить тільки виклик процедури, після якого стоїть оператор wait.

оператор_паралельного_виклику_процедури ::=
    [мітка :] оператор_виклику_процедури

Процедура не може мати формальних параметрів класу **variable**, тому що змінні не можуть бути видимим у будь-якомі місці, де можуть використовуватися паралельні оператори. Список чутливості оператора **wait** в еквівалентному процесі включає всі сигнали, які є фактичними параметрами процедури і мають режим **inout** чи **in**. Тільки такі сигнали можуть читатися процедурою.

Паралельні виклики процедури часто використовуються з процедурами, які можуть використовуватися в декількох місцях чи у різних моделях. Наприклад, припустимо, що у пакеті bit_vect_arith є процедура

```
procedure add(signal a, b : in bit_vector; signal result : out bit_vector);
```

Тоді цю процедуру можна викликати таким чином:

```
adder : bit_vect_arith.add (sample, old_accum, new_accum);
```

Такий виклик буде еквівалентним процесу

```
adder : process
begin
        bit_vect_arith.add (sample, old_accum, new_accum);
        wait on sample, old_accum;
end process adder;
```

5.4. Операторна частина оголошення об'єкта

У розділі 2.1 було вказано, що оголошення об’єкта може включати оператори для контролю за роботою об’єкта. Синтаксис оголошення об’єкта такий:

```
оголошення_об’єкта ::=
        entity ідентифікатор is
                заголовок_об’єкта
                розділ_оголошень_об’єкта
        [ begin
                операторна_частина_оголошення_об’єкта ]
        end [ просте_ім’я_об’єкта ] ;
операторна_частина_оголошення_об’єкта ::= { оператор_об’єкта }
оператор_об’єкта ::=
        паралельний_оператор_повідомлення
        | пасивний_оператор_паралельного_виклику_процедури
        | пасивний_оператор_процесу
```

Паралельні оператори, які розташовані в операторній частині оголошенні об’єкта, повинні бути пасивними, тобто вони не можуть містити ніяких операторів присвоювання сигналам (Це включає заборону присвоювання сигналам у вкладених процедурах чи процесах.) Результат цього правила – те, що такі процеси не можуть змінювати сигнали об’єкта і не можуть впливати на стан об’єкта. Однак, вони можуть контролювати його стан і часто використовуються, щоб повідомляти про помилкові робочі умови чи відслідковувати роботу об’єкта.

5.5. Нульова трансакція

У VHDL є засоби для моделювання виходів логічних схем, які можуть бути виключені (наприклад, драйвери з трьома станами). Оператор присвоювання сигналу може визначити, що ніяке значення не повинне бути присвоєне сигналу, тобто, що драйвер повинен бути відключений, а значення сигналу після виконання такої трансакції визначається його типом. Це робиться

за допомогою планування нульової (**null**) трансакції. Синтаксис опису елемента форми сигналу має такий вигляд (див. розділ 3.4):

елемент_форми_сигналу ::=

вираз_*значення_сигналу* [**after** вираз_*часу*]

| **null** [**after** вираз_*часу*]

Приклад планування нульової трансакції:

```
out <= null after 10 ns;
```

Слід відзначити, що нульова трансакція може використовуватися тільки з *захищеними (guarded)* сигналами. Це сигнали, клас яких явно вказаний при їх оголошенні. Існує два класи сигналів – регістр (**register**) та шина (**bus**). Зауважте, що захищені сигнали *обов'язково* мають тип, який дозволяє розв'язання конфліктів. Сигнали, клас яких явно не вказано, називають *незахищеними*.

Клас сигналу визначає, яким буде його значення, якщо всі його драйвери від'єднанні. Сигнал класу регістр після відключення драйверів зберігає останнє своє значення. Такі сигнали можуть використовуватися, щоб моделювати елементи пам'яті. Значення сигналу класу шина, коли від нього відключені всі драйвери, визначається функцією розв'язання конфліктів. Таким чином моделюють схеми з відкритим колектором, відкритим стоком та шини з трьома станами.

5.6. Захисний вираз блоку

В розділі 2.2.2 було вказано, що блок може мати захисний вираз. Такий вираз повинен мати тип boolean. Також в кожному блоці є вбудований сигнал guard, значення якого співпадає зі значенням захисного виразу. Значення цього сигналу можна перевірити в будь-якому місці блоку і, таким чином, дізнатися про значення захисного виразу блоку.

Значення захисного виразу визначає, чи будуть виконуватися оператори присвоювання значення сигналам зі словом guarded. Такі оператори присвоювання називають *захищеними*. Якщо значення захисного виразу true, то ці оператори будуть виконуватися. Якщо його значення false, то дії будуть залежати від того, присвоюється значення захищеному чи незахищеному сигналу. Для захищених сигналів в такому випадку буде плануватися нульова трансакція, тобто драйвер від'єднується:

```
if guard then
```

присвоювання сигналу
else
від'єднання драйвера
end if ;

Якщо захищений оператор використовується для присвоювання значення незахищеному сигналу, то, якщо значення захисного виразу false, ніякі дії не виконуються:

if guard **then**
присвоювання сигналу
end if ;

Зверніть увагу, що присвоювання значення захищеному сигналу за допомогою незахищеного оператора є помилковим.

З використанням захисного виразу можна легко описати D-тригер:

```
D_TRIGER:block (clk= '1' or clr= '1')
begin
        q<=guarded '0' when clr= '1'
                else d when clk= '1';
end block D_TRIGER;
```

Сигнал q в цьому прикладі повинен бути класу регістра.

Рекомендована література

1. IEEE Standard VHDL Language Reference Manual, p.290, New York, USA, The Institute of Electrical and Electronics Engineers, Inc., 2000.
2.*Армстронг Дж. Р.*, Моделирование цифровых систем на языке VHDL, с.175, М., Мир, 1991.
3.VHDL Golden Reference Guide, p.136, Ringwood, England, DOULOS, 1995.
4. *Ashenden P.J.*, The VHDL Cookbook, p.111, University of Adelaide, 1990.
3. Джехани Н., Язык Ада, с.552, М., Мир, 1988.

Зміст

ПЕРЕДМОВА 1

1. ОСНОВИ МОВИ VHDL 3

1.1. Структурний опис цифрової системи 3

1.2. Поведінковий опис цифрової системи 4

1.3. Модель дискретного часу 5

1.4. Приклад 7

1.5. Нормальні форми Бекуса – Наура 10

1.6. Лексичні Елементи 11
1.6.1. Коментарі 11
1.6.2. Ідентифікатори 11
1.6.3. Числа 11
1.6.4. Символи 12
1.6.5. Рядки 12
1.6.6. Рядки бітів 12

1.7. Типи та об'єкти даних 13
1.7.1. Цілий тип 14
1.7.2. Фізичний тип 14
1.7.3. Тип з плаваючою точкою 15
1.7.4. Тип перерахування 16
1.7.5. Підтипи 17
1.7.6. Індексний тип 18
1.7.7. Записи 20
1.7.8. Оголошення об'єктів даних 21
1.7.9. Атрибути 22

1.8. Вирази та операції 24

2. СТРУКТУРНИЙ ОПИС ЦИФРОВИХ СИСТЕМ НА МОВІ VHDL 26

2.1. Оголошення об'єкта 26

2.2. Архітектурне тіло об'єкта **29**
2.2.1. Оголошення сигналів 30
2.2.2. Блоки 30
2.2.3. Оголошення компонентів 32
2.2.4. Створення екземплярів компонентів 33
2.2.5. Оператор паралельної генерації 34

3. ПОВЕДІНКОВИЙ ОПИС СИСТЕМ В VHDL 37

3.1. Послідовні оператори **37**
3.1.1. Оператор присвоювання змінній 37
3.1.2. Умовний оператор 38
3.1.3. Оператор вибору 38
3.1.4. Оператор циклу 40
3.1.5. Нульовий (порожній) оператор 41
3.1.6. Оператор повідомлення 42

3.2. Підпрограми і пакети **42**
3.2.1. Процедури та функції 42
3.2.2. Перевантаження 46
3.2.3. Пакети і тіла пакетів 47
3.2.4. Використання пакетів та видимість імен 49

3.3. Процеси та оператор wait **50**

3.4. Присвоювання сигналу **53**

3.5. Паралельний оператор присвоювання сигналу **55**
3.5.1. Умовне присвоювання сигналу 55
3.5.2. Вибіркове присвоювання сигналу 57

4. ОРГАНІЗАЦІЯ МОДЕЛІ 59

4.1. Модулі проекту та бібліотеки **59**

4.2. Конфігурації **61**

4.3. Приклад проекту **65**

5. ДОДАТКОВІ ЗАСОБИ МОВИ VHDL 72

5.1. Розв'язання конфліктів **72**

5.2. Створення екземпляра об’єкта та конфігурації в архітектурному тілі 74

5.3. Паралельні оператори повідомлення та виклику процедури 75

5.4. Операторна частина оголошення об'єкта 76

5.5. Нульова трансакція 76

5.6. Захисний вираз блоку 77

РЕКОМЕНДОВАНА ЛІТЕРАТУРА 79